動画で上肢の動きを測る

マーカーレスモーションキャプチャー入門　理学療法士のための半日ワークショップ

場面

外来で、肩が上がりにくい患者さんを担当している

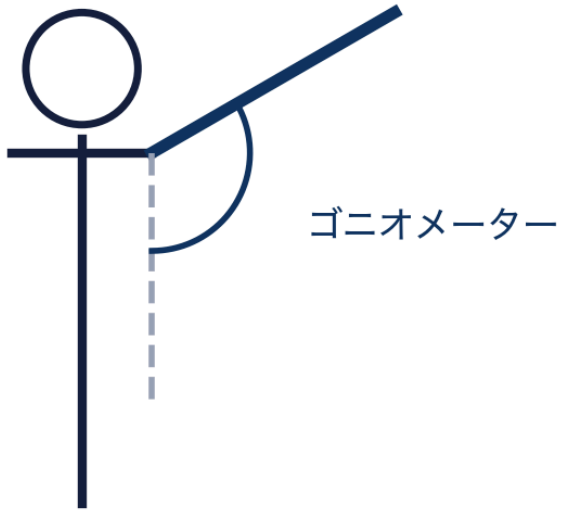
決めたいこと

この4週間で、肩は前より上がるようになったか

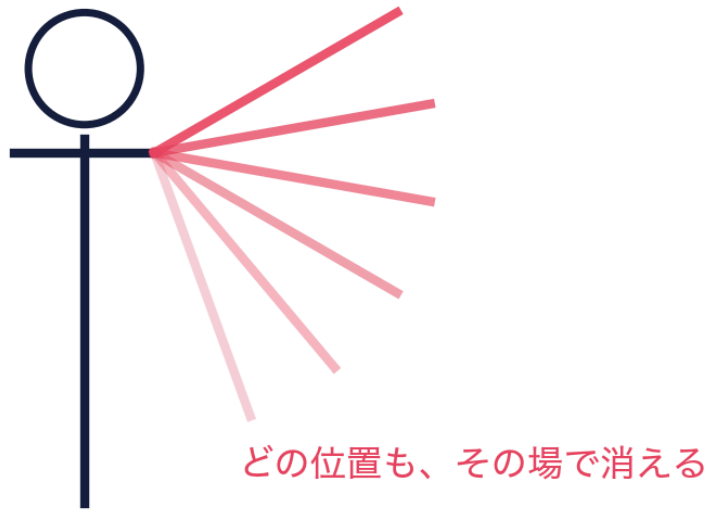
ゴニオメーターは止めて測る

- 止まった姿勢の角度は、よく測れる
- 動いている途中の角度は、測れない

止めた姿勢：測れる

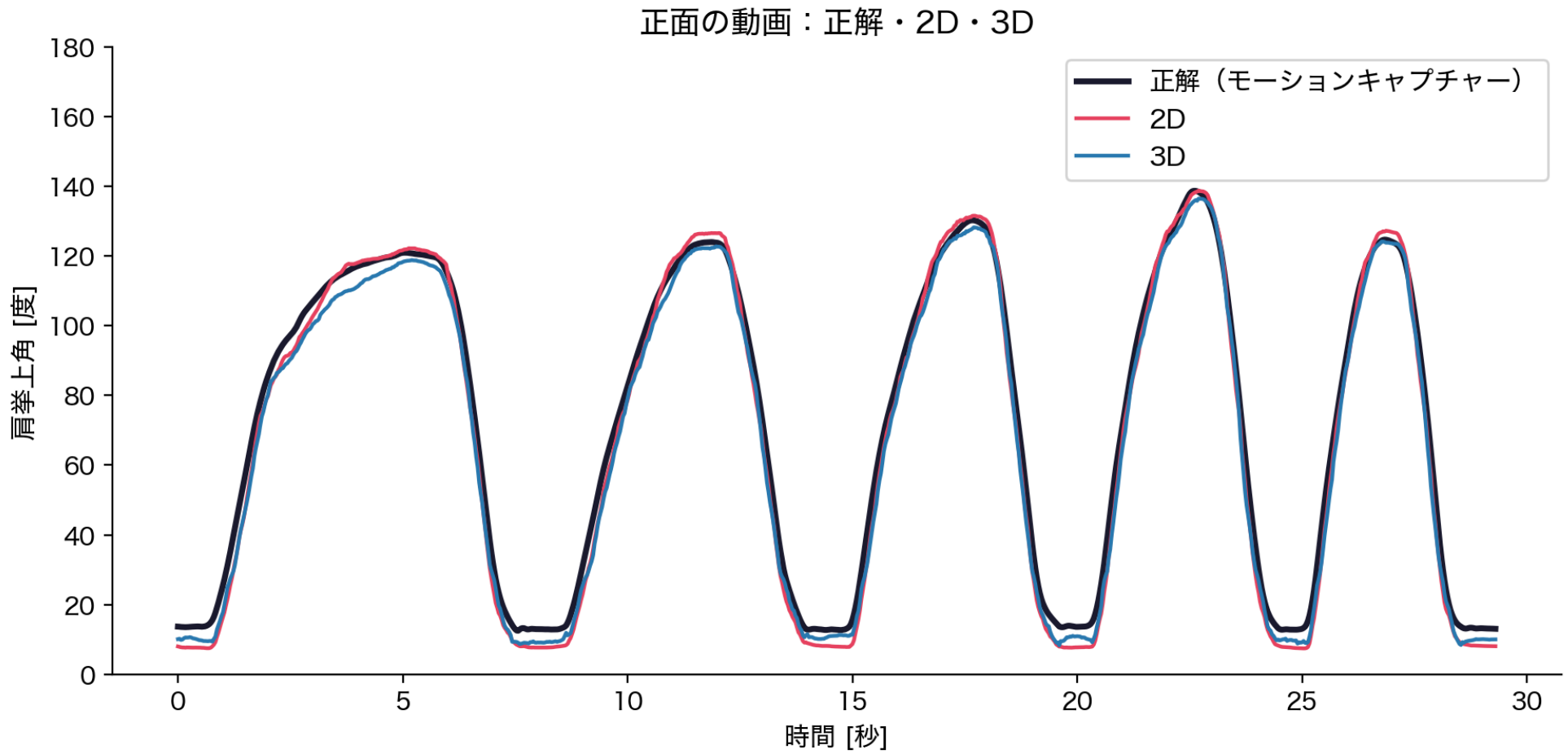


動いている途中：残らない



動画なら途中も残る

- 動画を撮れば、動きの全部のコマが残る
- 1コマごとに角度を出せば、動きが1本の線になる



手元のデータ①：正解つきの動画

- 同じ人の右腕の外転を、正面・斜め・横から撮った動画
- 同時に測ったモーションキャプチャーの角度を「正解」として使う

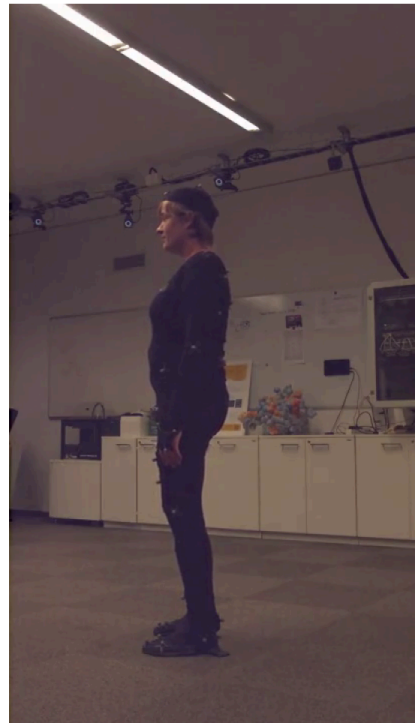
正面



斜め



横



データの出どころ

	中身
データ	REHAB24-6（Masaryk大学、2024年公開）
人	9番の人。患者ではない。理学療法士の指示で、正しく5回
長さ	正面と横は29.3秒（同時に撮影）、斜めは23.9秒。1秒30コマ
正解	16台のカメラのモーションキャプチャー

CC BY-NC 4.0。非営利の教育に使える

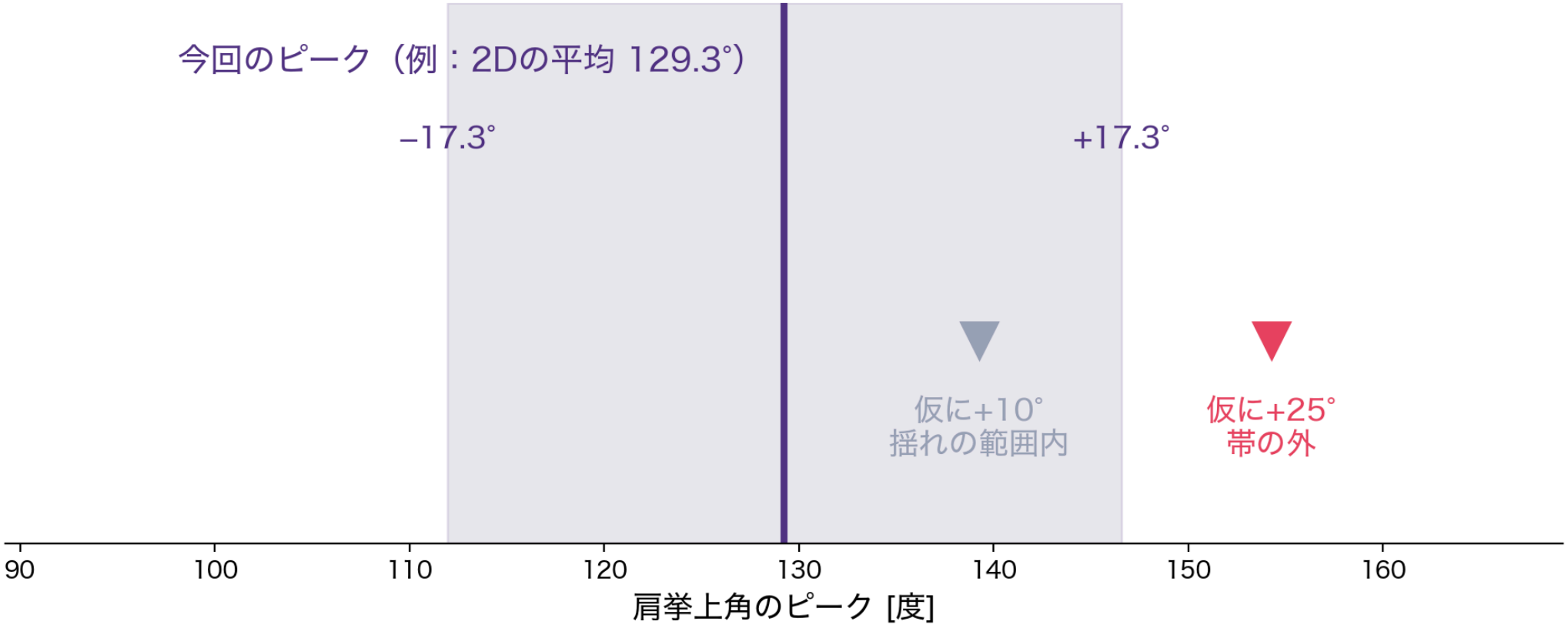
手元のデータ②：自分の動画

実習の最後に、自分のスマホで撮った動画も同じように測る

最後に出す答え

- 同じ撮り方で次に測ったとき、何度変われば「変わった」と言えるか

1回のピークどうしを比べるとき、帯の外なら「変わった」と言える



今日の問い

スマホ動画1本から、この上肢は良くなったと言えるか

今日の道筋

節	小さな問い
1	動画から何が取り出せるか
2	点から角度をどう出すか
3	実習①：自分で角度を出す
4	その数字は信じてよいか
5	撮る向きで数字は変わるか
6	どこまで、ではなく、どう動くか
7	生活の動き：飲水動作を見る
8	手指を見る

今日使うAI

	今日使うもの
AI（モデル）	MediaPipe（Google）。たくさんの画像で学び、写った人の関節の位置を推定する
動かす場所	Colab。ブラウザで開く、Googleの無料のサービス
準備	Googleアカウントだけ

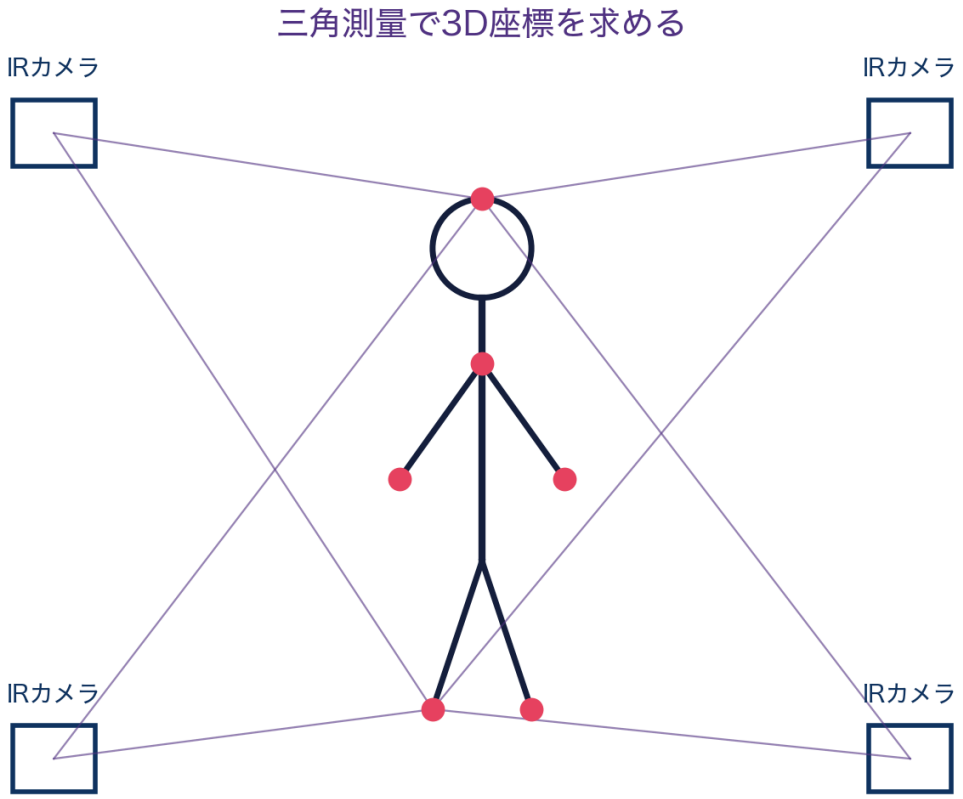
1. 動画から何が取り出せるか

角度を出すには、まず関節の位置が要る

マーカー式は体に印を貼る

- 反射する印を体に貼り、周りの赤外線カメラ数台で位置を測る

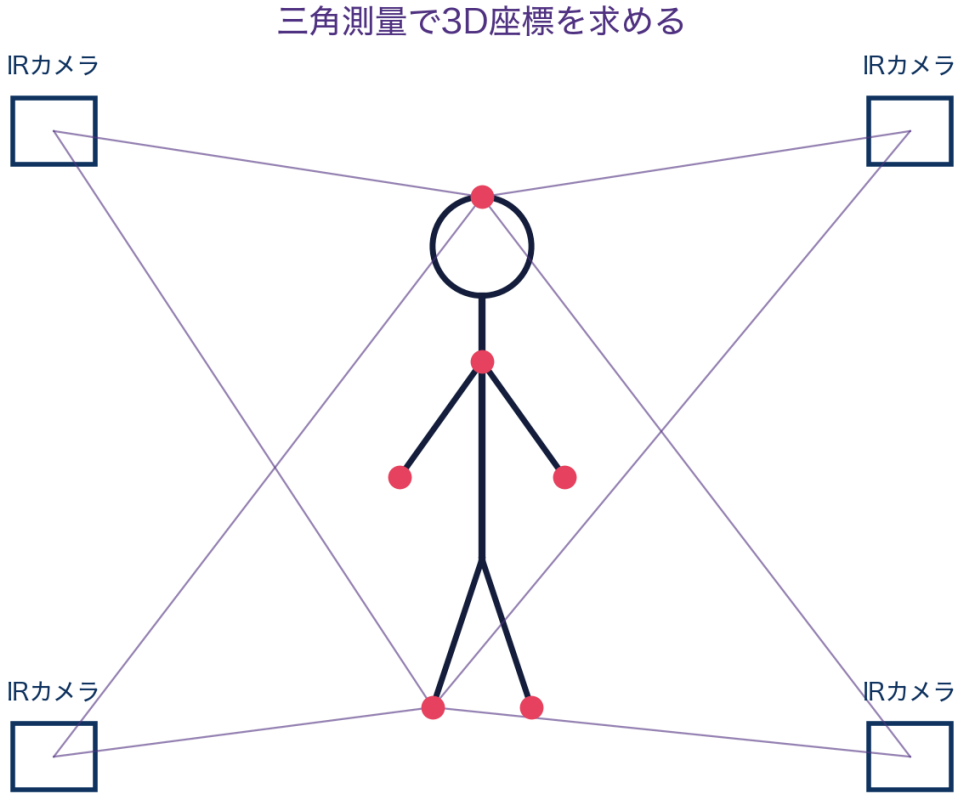
マーカー式MoCap：反射マーカー×赤外線カメラ



臨床ではそろえにくい

- 専用の部屋とカメラ、貼る手間、測る人が要る

マーカー式MoCap：反射マーカー×赤外線カメラ



マーカーレスは画像から推定

- 印を貼らず、写っている画像から関節の位置を推定する

マーカー式 と マーカーレス の発想の違い

マーカー式＝物理位置を測定

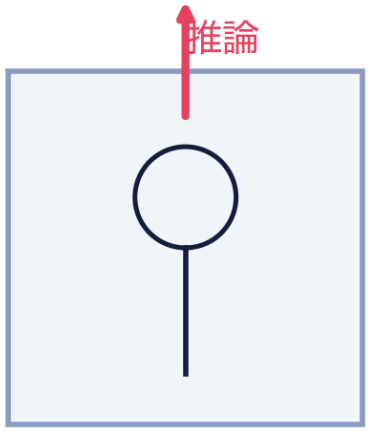
3D座標



反射マーカー
(物理的な点)

マーカーレス＝画像から推論

関節座標（確率的推定）

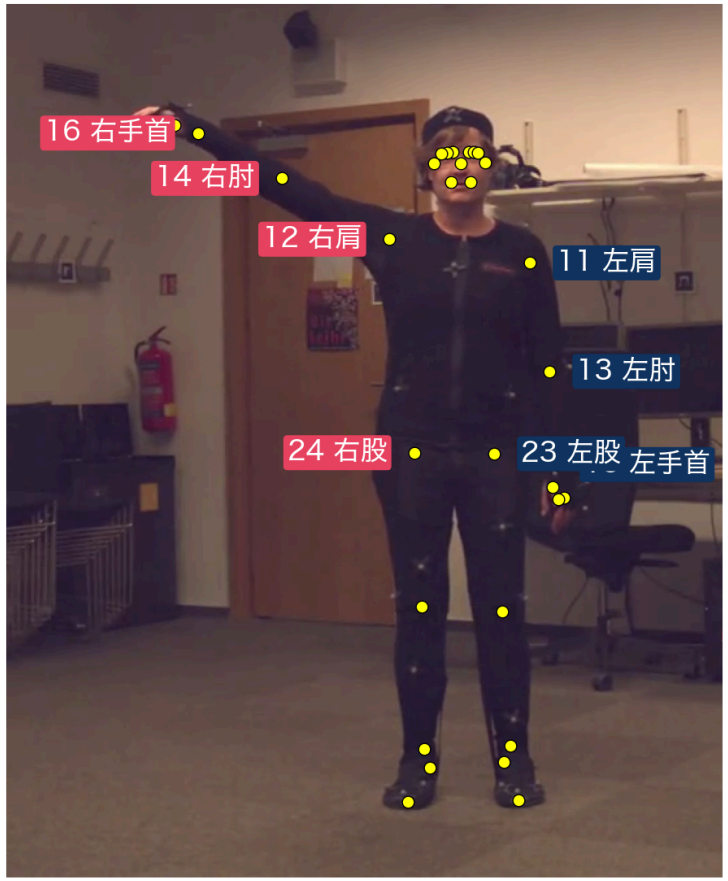


RGB画像のみ

今日使うのはMediaPipe

- Googleが無料で公開している道具。スマホでも動くほど軽い
- 動画を入れると、1コマごとに体の33点の位置が出る

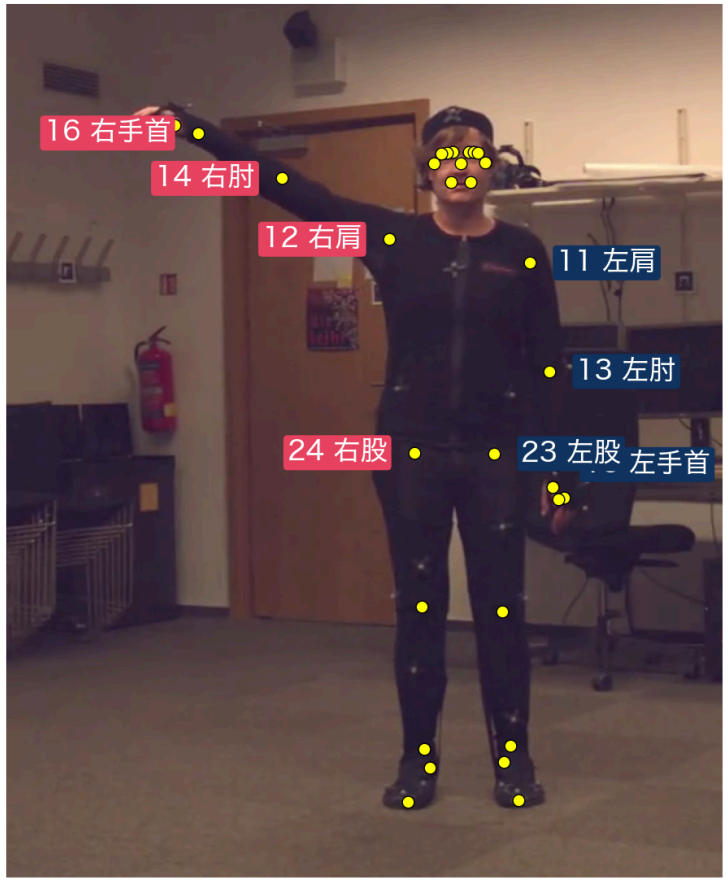
MediaPipeが出す33点（正面の動画、4秒目）



33点には番号がある

- 今日使うのは、11・12（肩）、13・14（肘）、16（右手首）、23・24（股）

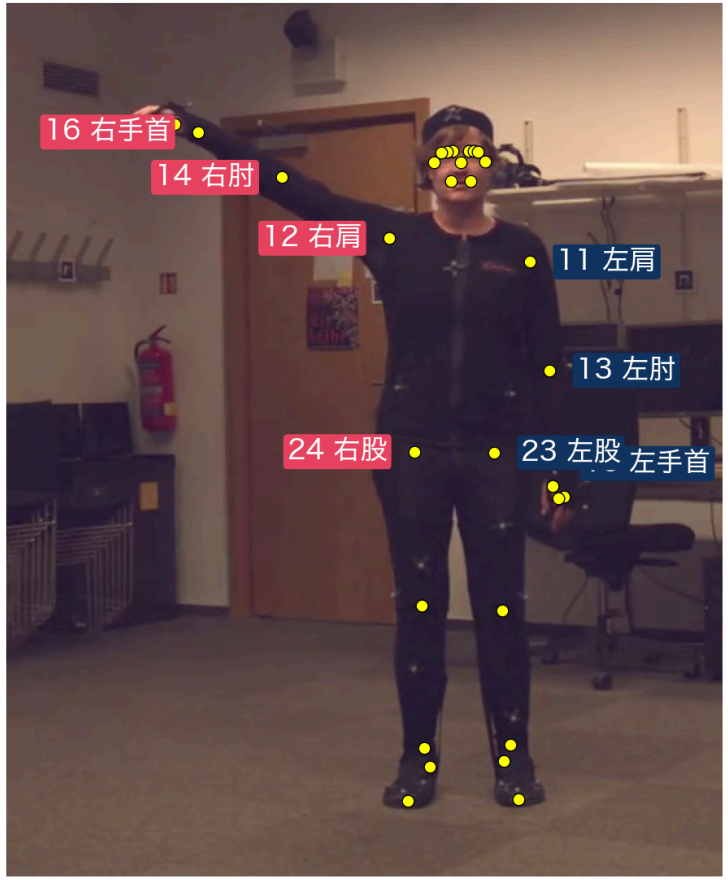
MediaPipeが出す33点（正面の動画、4秒目）



2Dの点：画像の中の位置

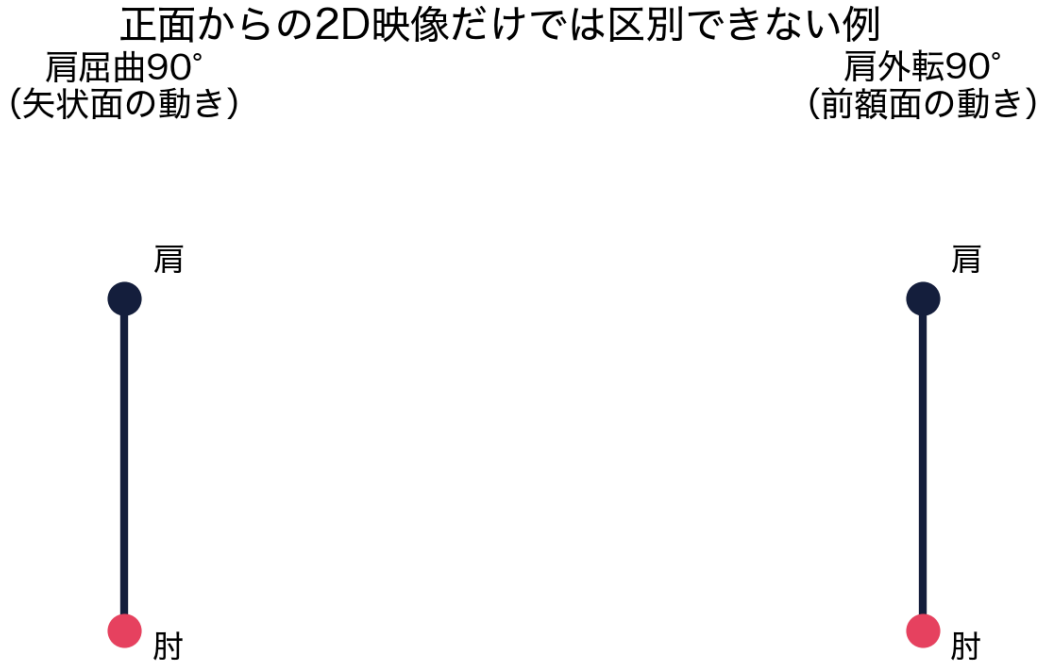
- 画像の左上を原点にした、横と縦の位置（ピクセル）
- 写っているとおりの位置なので、奥行きは分からない

MediaPipeが出す33点（正面の動画、4秒目）



カメラ1台では奥行きが分からない

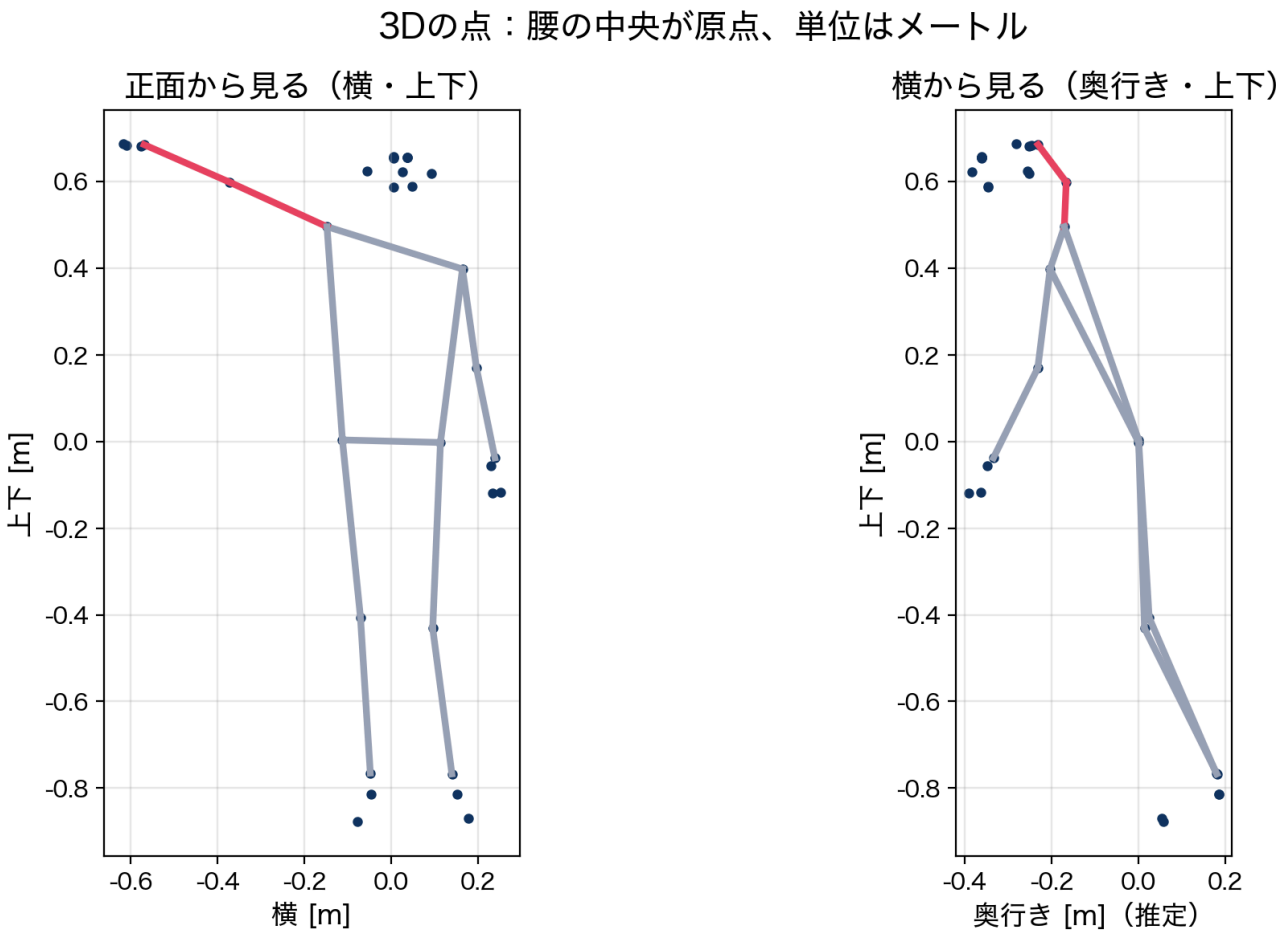
- 正面から見ると、前に出した腕と横に出した腕が同じに見える



→ 正面カメラの2D画像では同じに見え、区別できない

3Dの点：奥行きまで推定した位置

- 腰の中央を原点にした、横・上下・奥行きの位置（メートル）
- 奥行きは、人の体の形を学んだモデルが推定した値



MediaPipeは、1回の処理で2Dの点と3Dの点を両方出す

2. 点から角度をどう出すか

33点の位置は出た。では、点からどうやって角度を出すか

肩挙上角を振り返る

- 腕を下ろした位置が0°、真上が180°

肩挙上角の3態

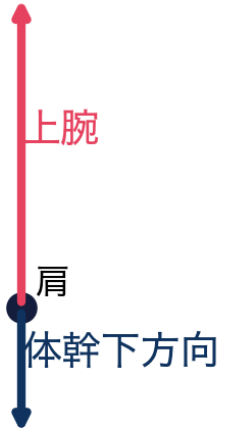
挙上0°



挙上90°



挙上180°



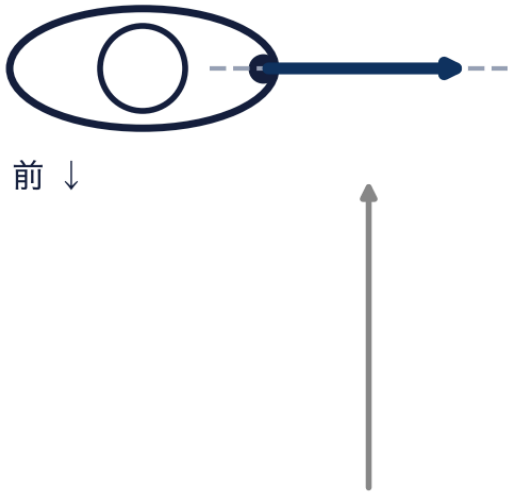
撮る向きで読み方が変わる

- 真横から撮れば屈曲、正面から撮れば外転として読む
- 今日の動画は外転を正面から撮っている

点線が動きの面。カメラを動きの面に向かい合わせる

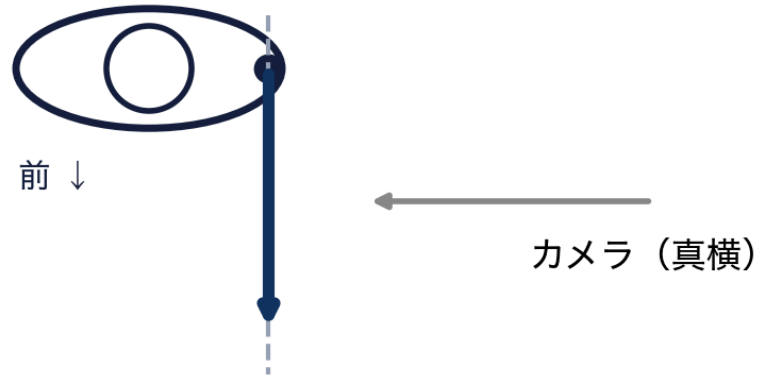
外転：正面から撮る

体（上から見た図）



屈曲：真横から撮る

体（上から見た図）



ゴニオメーターを点に当てる

- 軸は右肩、固定アームは体幹の線、移動アームは上腕の線
- 体幹の線は、両肩の中点から両股の中点へ

日本の関節可動域表示の基本軸は「肩峰を通る床への垂直線」。今日は、体が傾いても体に沿う体幹の線を使う

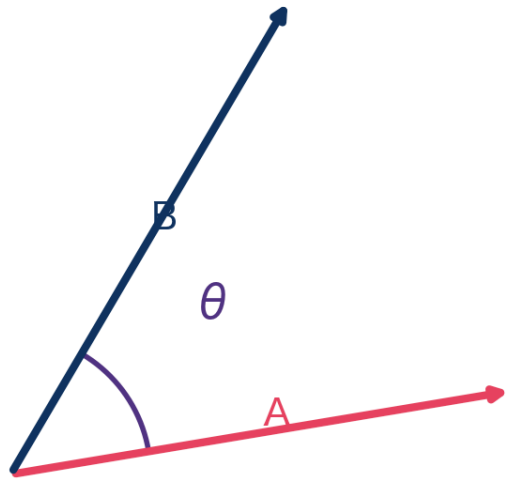
2Dの点で肩挙上角を測る（4秒目）



2本の線は2本の矢印

- 点から点へ引いた矢印を「ベクトル」と呼ぶ
- 角度は、2本の矢印のなす角 θ

内積とベクトルのなす角 θ



$$\cos\theta = A \cdot B / (|A||B|)$$

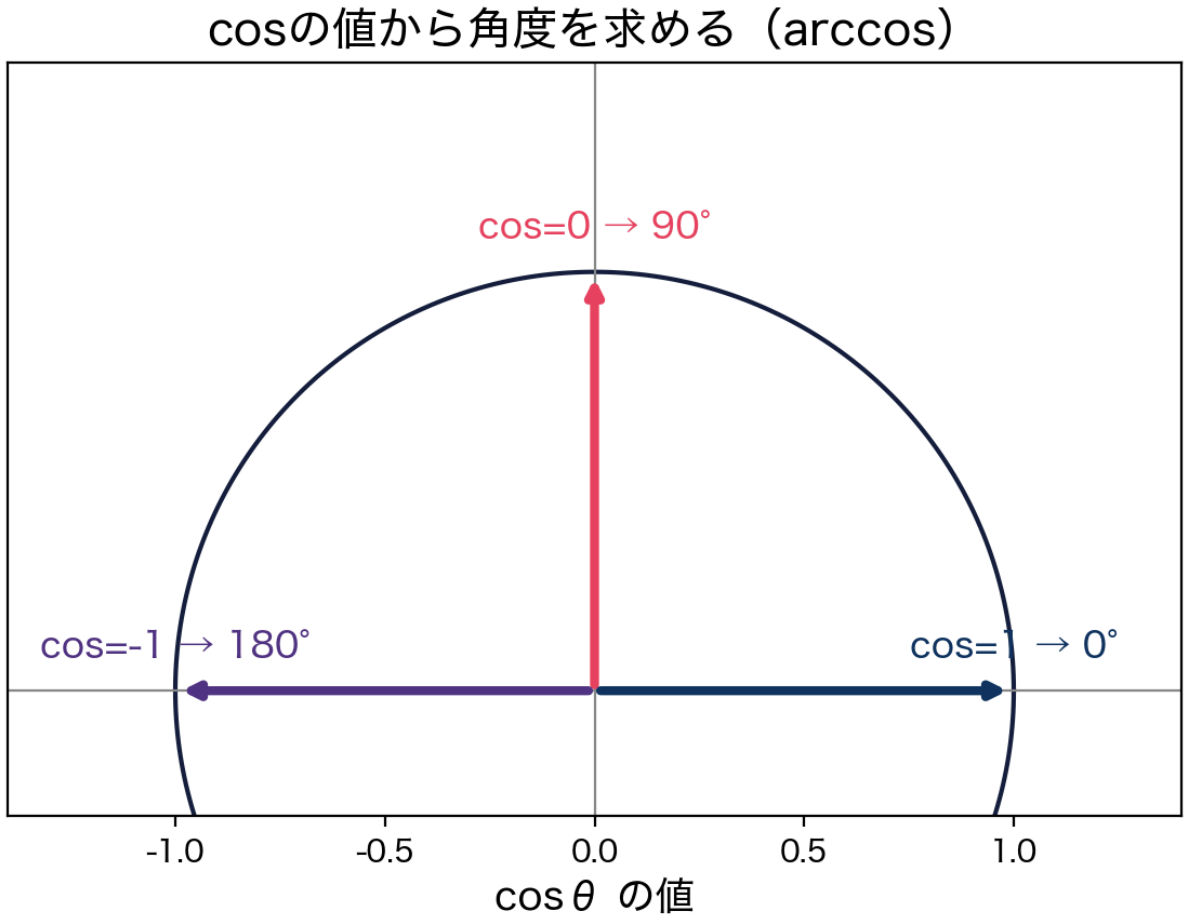
矢印2本から角度を出す式

$$\cos\theta = \frac{\vec{a}\cdot\vec{b}}{|\vec{a}|\,|\vec{b}|}$$

- $\vec{a}$：上腕の矢印、$\vec{b}$：体幹の矢印
- 分子の $\vec{a}\cdot\vec{b}$（内積）は、2本の向きのそろい具合
- 分母で割ると、矢印の長さが結果に影響しなくなる

cosの値から角度に戻す

- 向きがそろえば1で0°、直角なら0で90°、逆向きなら−1で180°



2Dでも3Dでも同じ式

矢印が2つの数でも3つの数でも、内積と長さは同じように計算できる

実データの1コマで測る

- 正面の動画の4秒目。2Dの点で測ると118.1°

2Dの点で肩挙上角を測る（4秒目）



2Dと3Dで値が違う？

点	4秒目の肩挙上角
2D（画像の中の位置）	118.1°
3D（奥行きまで推定）	111.3°

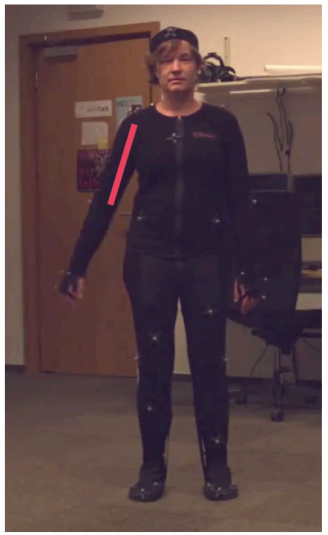
- 2Dは、画像に映ったままの角度
- 3Dは、推定した奥行きも入れた角度

どちらが本当の角度に近いかは、正解と比べて確かめる

全部のコマで同じ計算をする

- 880コマすべてで同じ計算をして、時間の順に並べる

1.0秒
16.4°

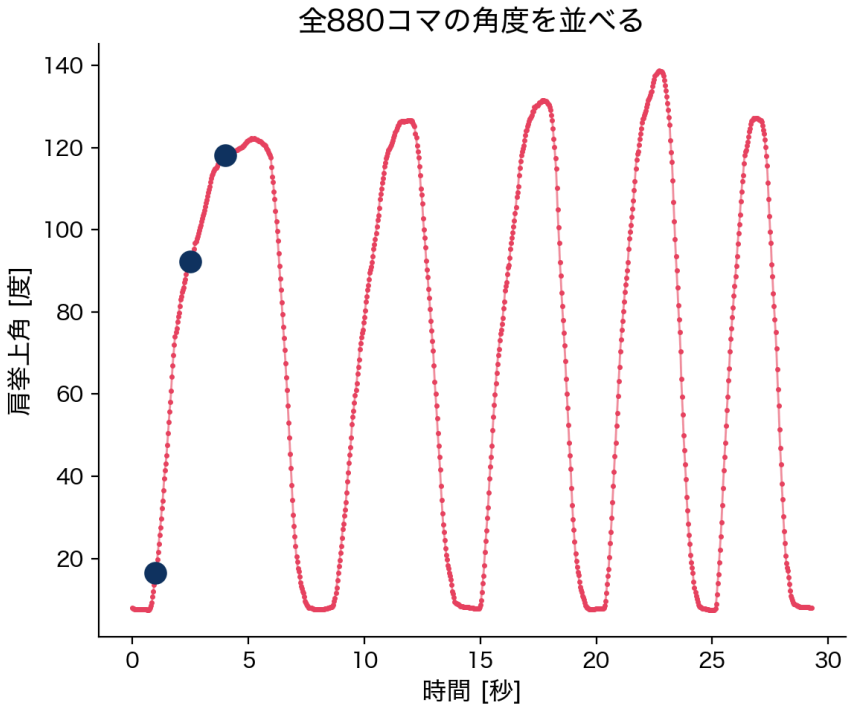


2.5秒
92.2°



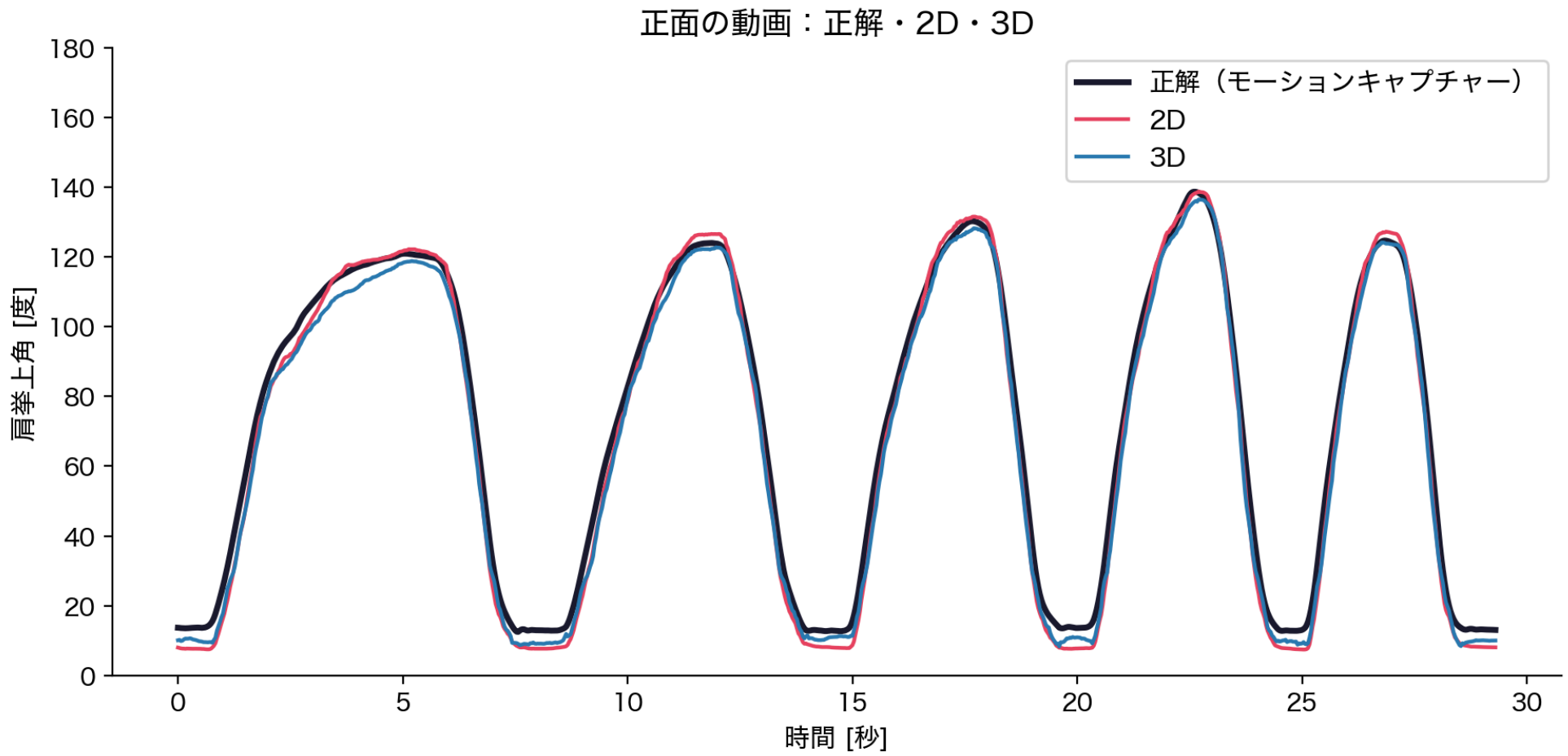
4.0秒
118.1°





動きが1本の線になる

- 点を線で結ぶと、挙上の繰り返しが山として見える



3. 実習①：自分で角度を出す

計算の流れは分かった。ここからは、同じ計算を自分のColabで動かす

コードはコンピューターへの手順書

- 評価の手順書と同じく、上から順に1行ずつ進む

評価の手順書（人が読む）	コード（コンピューターが読む）
1. 端座位をとってもらう	`points2d, points3d, fps = track_pose(...)`
2. 肩にゴニオメーターを当てる	`angle = shoulder_elevation(...)`
3. 角度を読んで記録する	`print(angle)`

どちらも「上から順に1行ずつ」進む

いちばん短いコード

```
print('こんにちは')
```

- `print(…)` は、かっこの中を表示する命令
- `'` で囲んだものは、そのままの文字として扱われる

実行すると下に出る

```
print('こんにちは')
```

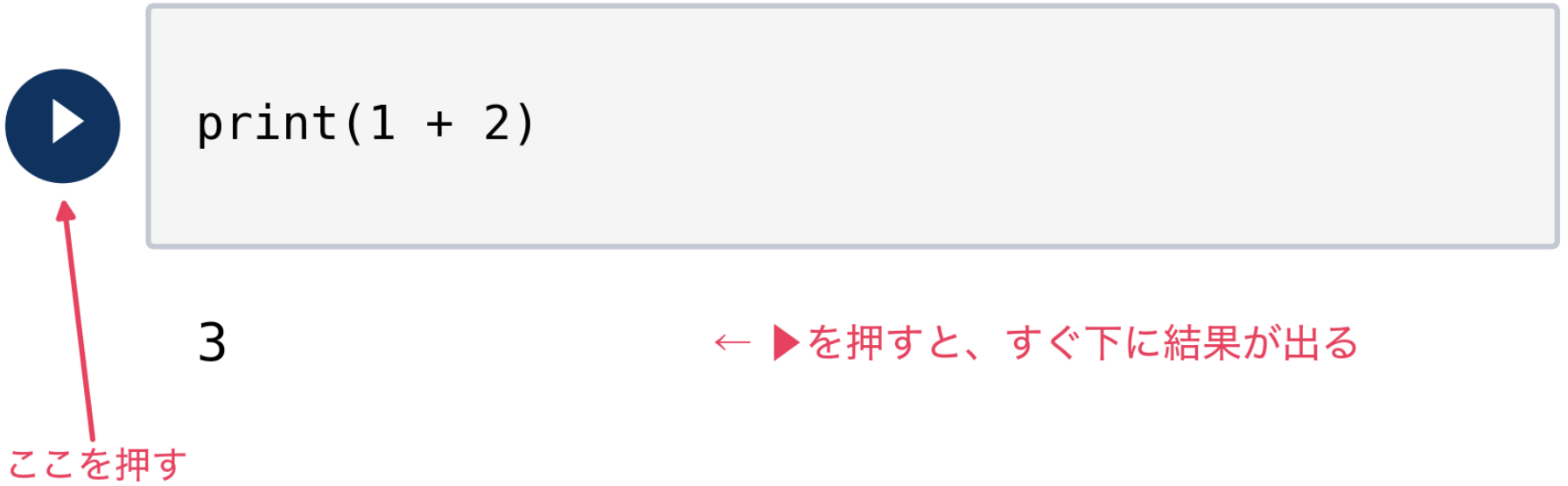
```
こんにちは
```

コードを動かすことを「実行する」と呼ぶ

セルは押すと動く箱

- Colabでは、コードを「セル」という箱に入れて、1つずつ実行する

セル：コードを入れる箱



ノートを開く

1. 配られたリンクの「Colabで開く」を押し、Googleアカウントでログインする
2. 左上の［ドライブにコピー］を押す
3. 「0. 最初の1行」のセルの▶️を押し、「こんにちは」が出るのを確かめる

約束

上のセルから順に、全部押す。下のセルは、上のセルで作ったものを使う

道具箱を開ける

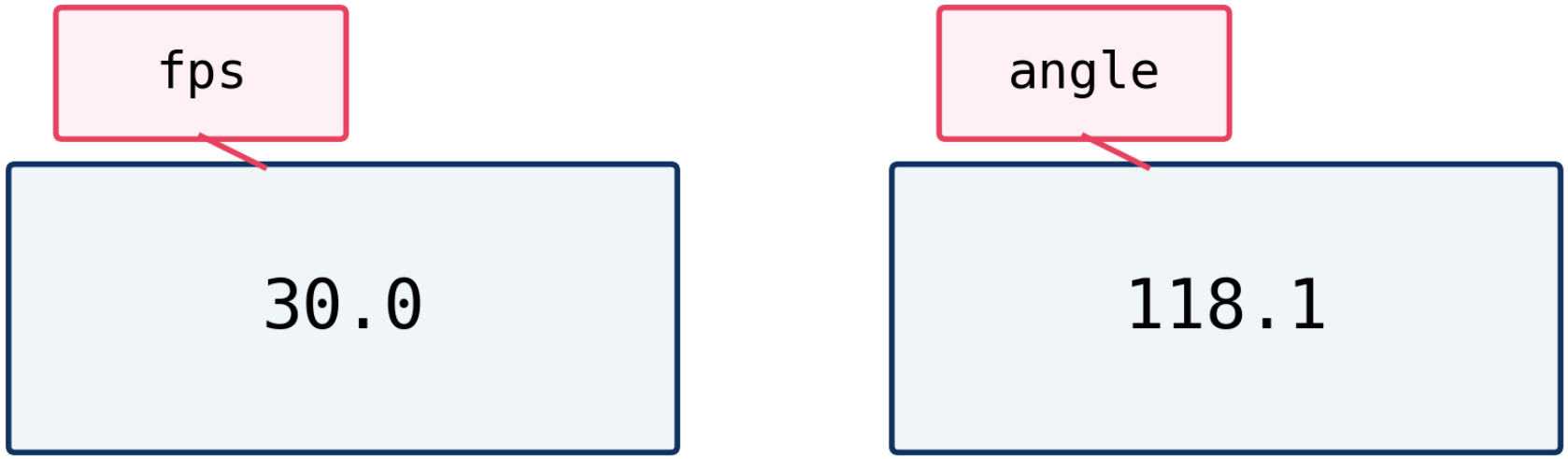
```
!pip install -q mediapipe==0.10.35 matplotlib-fontja
import numpy as np
import matplotlib.pyplot as plt
import matplotlib_fontja
from scipy.signal import find_peaks, savgol_filter
print('道具の準備ができました')
```

- `import` は、計算やグラフの道具が入った箱を開ける命令
- 1行目は、MediaPipeと日本語の道具を入れる行。先頭の `!` はColabへの頼みごと
- `as np` は短い呼び名。このあとは `np.` と書いて道具を使う

スライドの数は講師の手元で出したもの。Colabでは小数点以下や山の数が少しずれることがある

変数は値に付けた名札

- 値に名前を付けて取っておくと、あとで名前を書くだけで取り出せる



名前（名札）を書けば、中の値を何度でも取り出せる

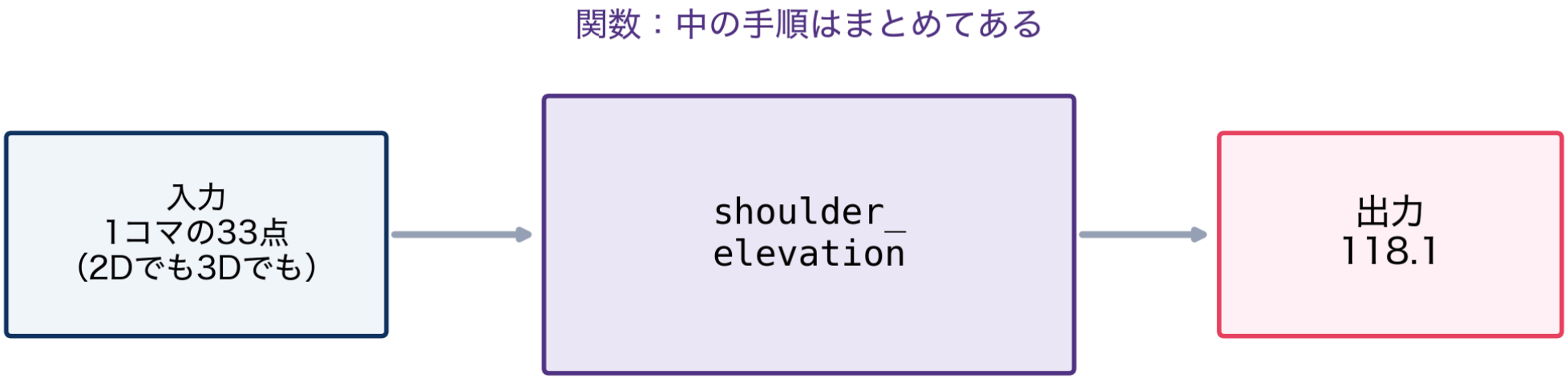
最初の変数：データの置き場所

```
URL = 'https://k0heiun0.github.io/pt_seminar/data/rehab24/'
!wget -q {URL}front.mp4 {URL}side.mp4 {URL}oblique.mp4
!wget -q {URL}front_truth.csv {URL}side_truth.csv {URL}oblique_truth.csv
!wget -q {URL}front_truth_wrist.csv
print('データを持ってきました')
```

- 1行目：データの置き場所に、`URL` という名札を付ける
- `=` は「等しい」ではなく「右の値に左の名前を付ける」

関数は入力を結果に変える機械

- 中の手順をまとめておき、入力を渡すと結果が返ってくる



使う人は「何を入れると、何が出るか」だけ知っていればよい

関数の形

```
def 関数の名前(入力):
    手順
    return 結果
```

- `def` で始め、 `return` で結果を返す
- 手順は字下げして、かたまりにする

この枚は形の説明で、ノートにはない

道具の関数は読み飛ばしてよい

関数	役割	入力	出力
`show_frame`	1コマを表示する	動画の名前、何秒目か	画像
`track_pose`	全コマで33点を出す	動画の名前	2Dの点、3Dの点、1秒のコマ数
`draw_points`	1コマに33点を重ねる	動画の名前、秒、2Dの点、コマ数	画像

ノートの「3. 道具の関数を作る」。中身は読まず、役割・入力・出力だけ読めばよい

押しても何も出ない

関数は、作っただけでは動かない。呼んだときに動く

関数を呼ぶ

```
show_frame('front.mp4', 4)
```

- `関数の名前(入力)` と書くと、関数が動く
- 入力は、動画の名前と何秒目かの2つ
- 前に「命令」と呼んだ `print(…)` も、同じ形の関数

体の33点を出す

```
points2d, points3d, fps = track_pose('front.mp4')
print('コマ数', len(points2d))
print('1秒あたりのコマ数', fps)
```

```
コマ数 880
1秒あたりのコマ数 30.0
```

返ってくる3つの結果に、`points2d`・`points3d`・`fps`という名札を付けた

1コマの点を描く

```
draw_points('front.mp4', 4, points2d, fps)
print('右肩 2D', np.round(points2d[120][12], 1))
print('右肩 3D', np.round(points3d[120][12], 3))
```

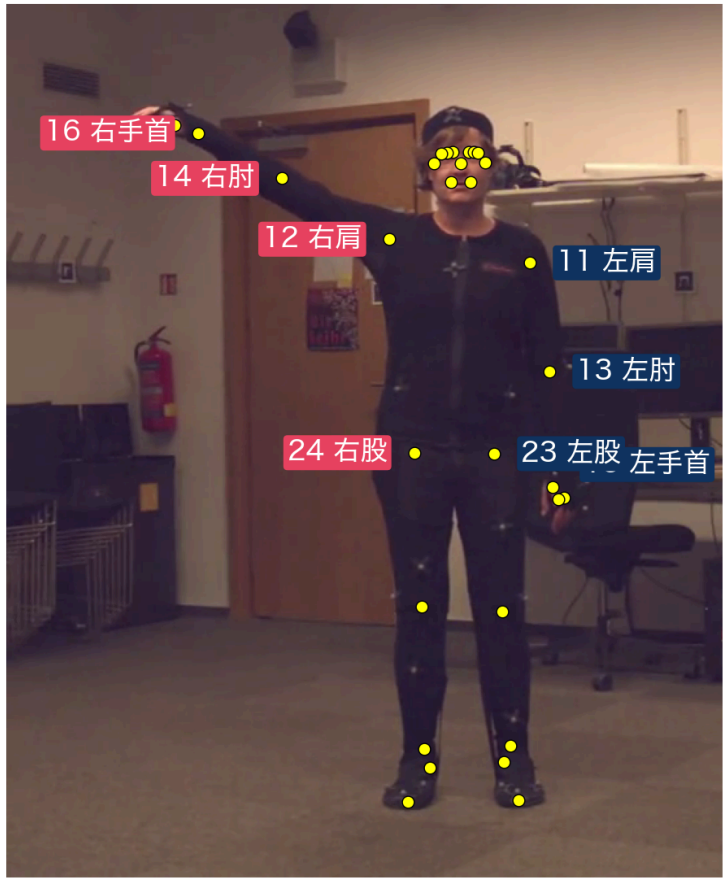
```
右肩 2D [548.1 235.6]
右肩 3D [-0.148 -0.495 -0.171]
```

`points2d[120][12]` は、120コマ目（4秒目）の12番の点（右肩）。番号は0から数える

出てくる画像

- 2Dは2つの数（ピクセル）、3Dは3つの数（メートル）

MediaPipeが出す33点（正面の動画、4秒目）



2本の矢印のなす角の関数

```
def angle_between_vectors(v1, v2):
    v1 = v1 / np.linalg.norm(v1)
    v2 = v2 / np.linalg.norm(v2)
    cos_angle = np.clip(np.dot(v1, v2), -1.0, 1.0)
    return np.degrees(np.arccos(cos_angle))
```

- 役割：2本の矢印のなす角を出す（2節の式そのもの）
- 入力：2本の矢印 `v1` と `v2`　出力：角度（度）
- 中身は読み飛ばしてよい

肩挙上角の関数

```
def shoulder_elevation(p):
    # 1. どの点を使うか（番号は0から数える）
    shoulder, elbow = p[12], p[14]
    mid_shoulder = (p[11] + p[12]) / 2
    mid_hip = (p[23] + p[24]) / 2
    # 2. 2本の線を作る
    upper_arm = elbow - shoulder          # 移動アーム
    trunk = mid_hip - mid_shoulder        # 固定アーム
    # 3. なす角を返す
    return angle_between_vectors(upper_arm, trunk)
```

3つのかたまりで読む

1. どの点を使うか：右肩12番、右肘14番、両肩と両股の中点
2. 2本の線を作る：点から点を引くと、矢印になる
3. なす角を返す：さっきの関数に2本の矢印を渡す

`#` から行末までは、人のためのメモ（コメント）。実行されない

ゴニオメーターと同じ当て方

- 軸は右肩、固定アームは体幹の線、移動アームは上腕の線

2Dの点で肩挙上角を測る（4秒目）



2Dと3Dの点を同じ関数に渡す

```
print('4秒目の肩挙上角 2D', round(shoulder_elevation(points2d[120]), 1))
print('4秒目の肩挙上角 3D', round(shoulder_elevation(points3d[120]), 1))
```

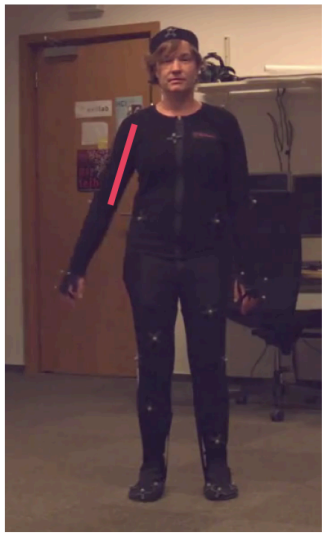
```
4秒目の肩挙上角 2D 118.1
4秒目の肩挙上角 3D 111.3
```

2節で図を描いて出した値と、同じ値が返ってきた

880コマぶん繰り返したい

- 1コマで測れた。同じ測定を、全部のコマで繰り返す

1.0秒
16.4°



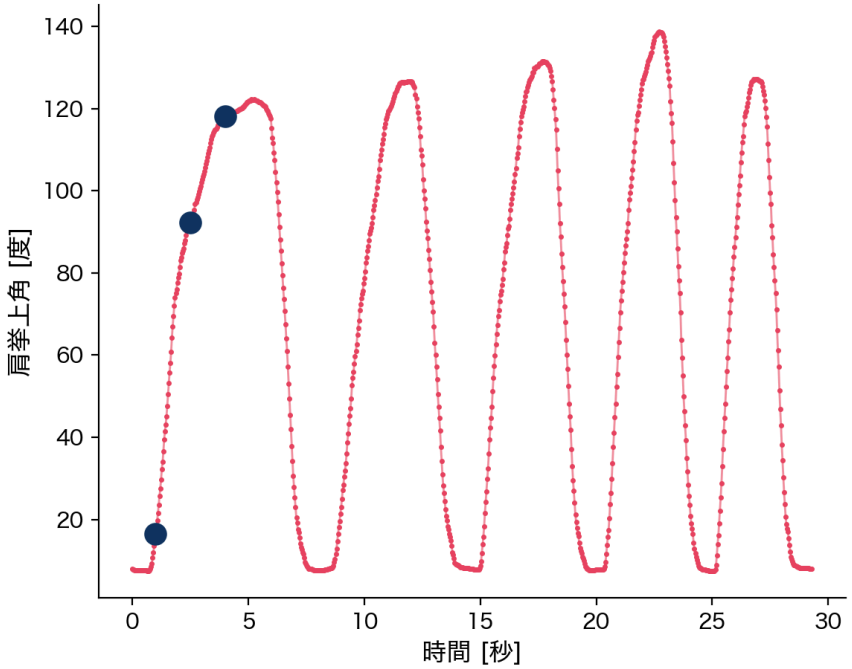
2.5秒
92.2°



4.0秒
118.1°



全880コマの角度を並べる



繰り返しの命令for

```
angle2d = []
angle3d = []
for k in range(len(points2d)):
    angle2d.append(shoulder_elevation(points2d[k]))
    angle3d.append(shoulder_elevation(points3d[k]))
angle2d = np.array(angle2d)
angle3d = np.array(angle3d)
print('2Dの最大', round(np.nanmax(angle2d), 1))
print('3Dの最大', round(np.nanmax(angle3d), 1))
```

```
2Dの最大 138.7
3Dの最大 136.5
```

1行ずつ読む

- `angle2d = []`：空の入れ物を用意する
- `for k in range(len(points2d)):`：`k`を0、1、2…879と変えながら、字下げした行を繰り返す
- `.append(…)`：測った角度を入れ物の最後に足す

正解を読み込む

```
truth = np.loadtxt('front_truth.csv', delimiter=',', skiprows=1, usecols=2)
print('正解の最大', round(truth.max(), 1))
```

```
正解の最大 138.7
```

- 正解のファイルから、3列目（肩挙上角）だけを読み込む
- `skiprows=1` で1行目の列の名前を飛ばし、`usecols=2` で3列目だけを使う

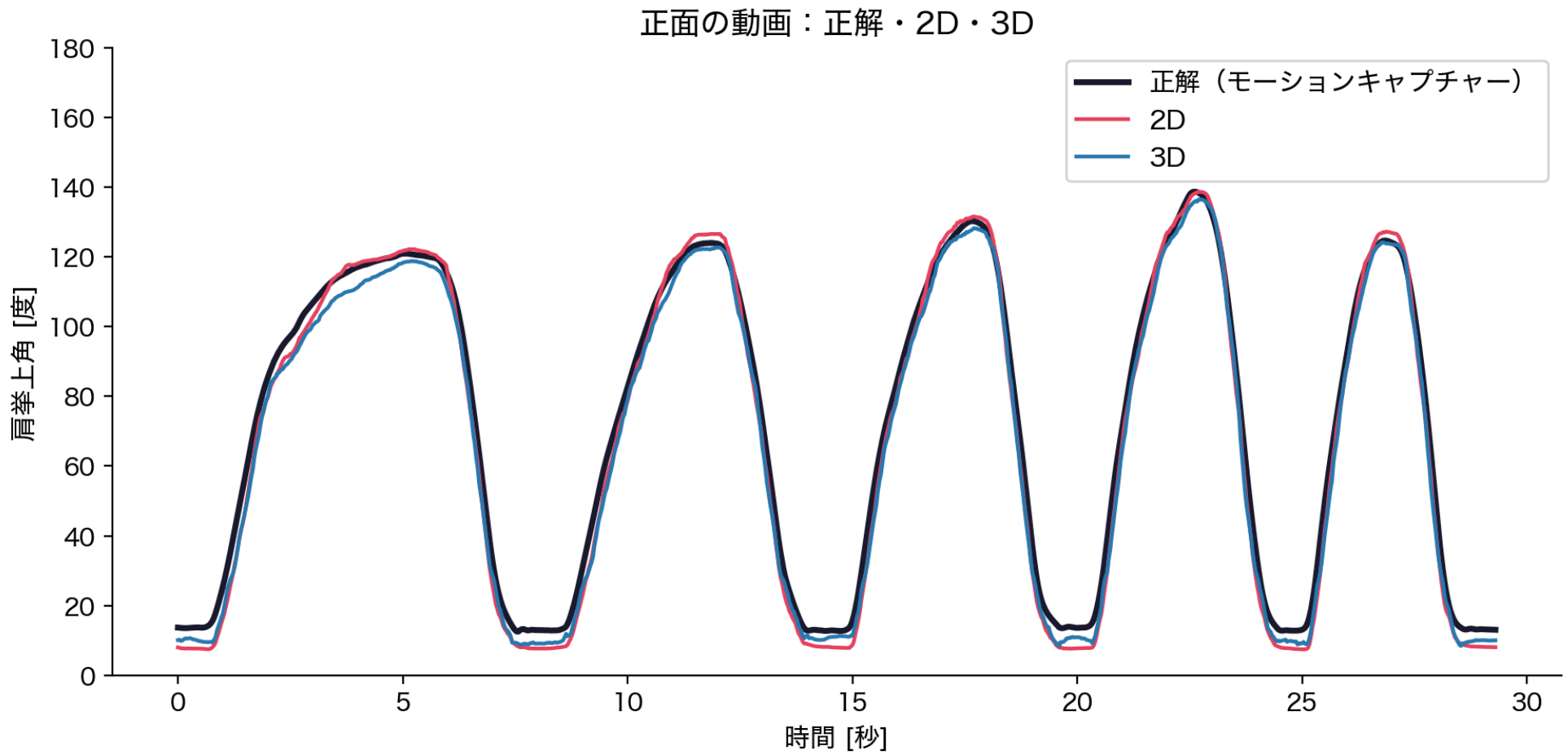
グラフを描く関数

```
time = np.arange(len(angle2d)) / fps
plt.plot(time, truth, label='正解', color='black')
plt.plot(time, angle2d, label='2D')
plt.plot(time, angle3d, label='3D')
plt.xlabel('時間 [秒]')
plt.ylabel('肩挙上角 [度]')
plt.legend()
plt.show()
```

`plt.plot(横, 縦, label=名前)` で線を引き、凡例を付けて表示する

出てくるグラフ

- 正面から撮れば、2Dも3Dも正解とほぼ重なる



正解との差を数にする

```
print('2Dと正解の差', round(np.nanmean(np.abs(angle2d - truth)), 1))
print('3Dと正解の差', round(np.nanmean(np.abs(angle3d - truth)), 1))
```

```
2Dと正解の差 4.3
3Dと正解の差 4.4
```

コマごとの差の大きさを平均した。`np.nanmean`は空欄を飛ばして平均する

困ったとき

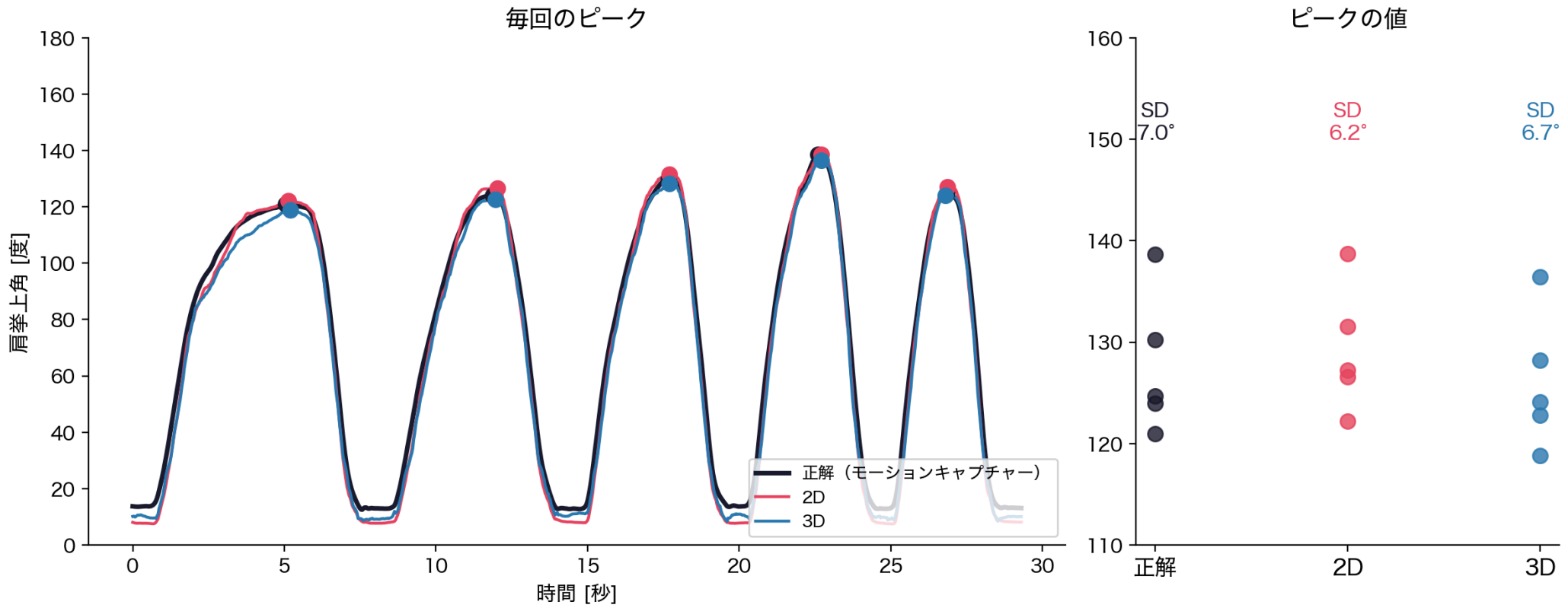
出たもの	よくある理由	直し方
`NameError`	上のセルを押していない	［ランタイム］→［すべてのセルを実行］
`mediapipe` が入らない	Colabの版が変わった	1のセルの1行目を `!pip install -q mediapipe matplotlib-fontja` にする
グラフの文字が□	1のセルを押していない	1のセルから押し直す

4. その数字は信じてよいか

角度は出せた。では、その数字は信じてよいか

同じ挙上なのに、ピークは毎回違う？

- 正しく行った5回なのに、ピークは毎回少しずつ違う



ピークを数えてSDを出す

```
def peak_summary(angle):
    angle = np.nan_to_num(angle)     # 空欄は0として山を探す
    peaks, _ = find_peaks(angle, height=90, prominence=30)
    values = angle[peaks]
    sd = values.std(ddof=1)
    return len(peaks), values.mean(), sd, 1.96 * np.sqrt(2) * sd

for name, angle in [('正解', truth), ('2D', angle2d), ('3D', angle3d)]:
    n, mean, sd, mdc = peak_summary(angle)
    print(name, '回数', n, '平均', round(mean, 1), 'SD', round(sd, 1),
          'MDC95', round(mdc, 1))
```

出てくる数

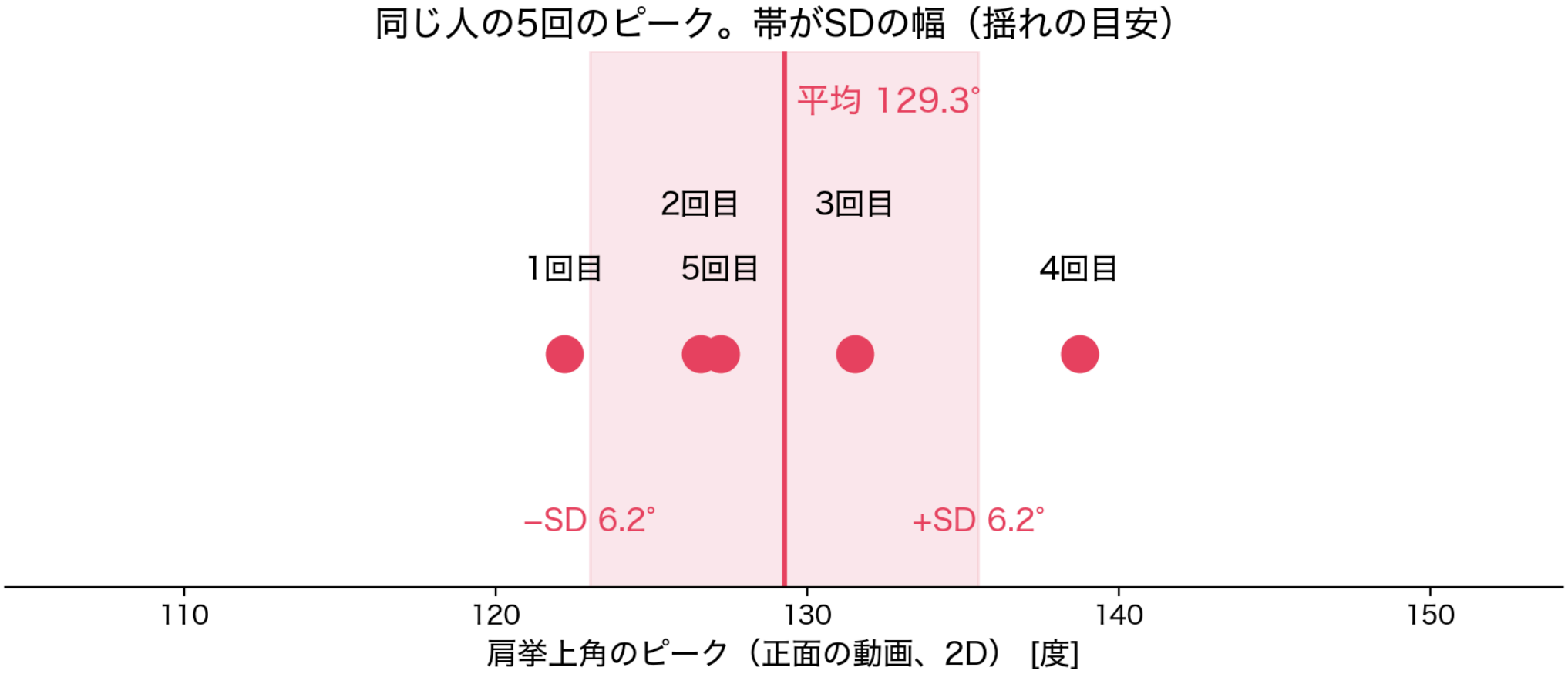
```
正解 回数 5 平均 127.7 SD 7.0 MDC95 19.4
2D 回数 5 平均 129.3 SD 6.2 MDC95 17.3
3D 回数 5 平均 126.1 SD 6.7 MDC95 18.6
```

- `find_peaks` は山を探す道具。90°を超えた山を1回と数える
- 正解でも、5回のピークのSDは7.0°ある

正解のピークも同じくらい揺れる。揺れの多くは、本人の動きから来ている

測定の揺れを表す数：SEM

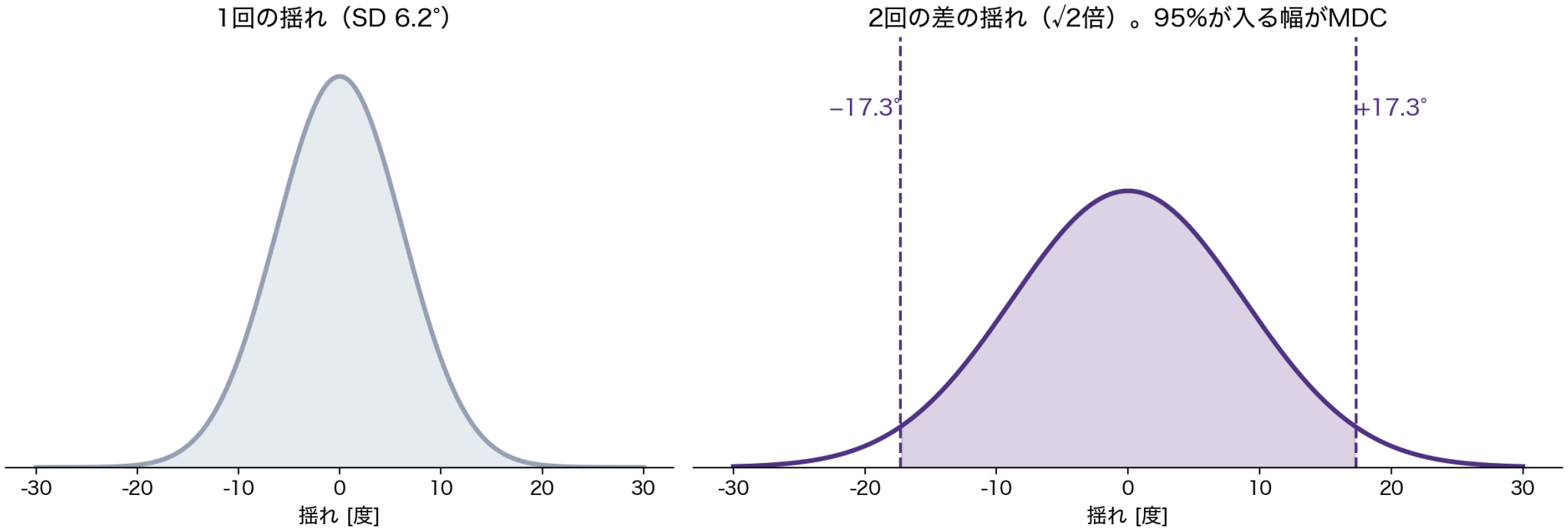
- 同じものを繰り返し測ったときの値の揺れ。測定誤差（standard error of measurement）と呼ぶ
- 同じ人の繰り返しの標準偏差（SD）が、その目安になる



変わったと言える最小の幅：MDC

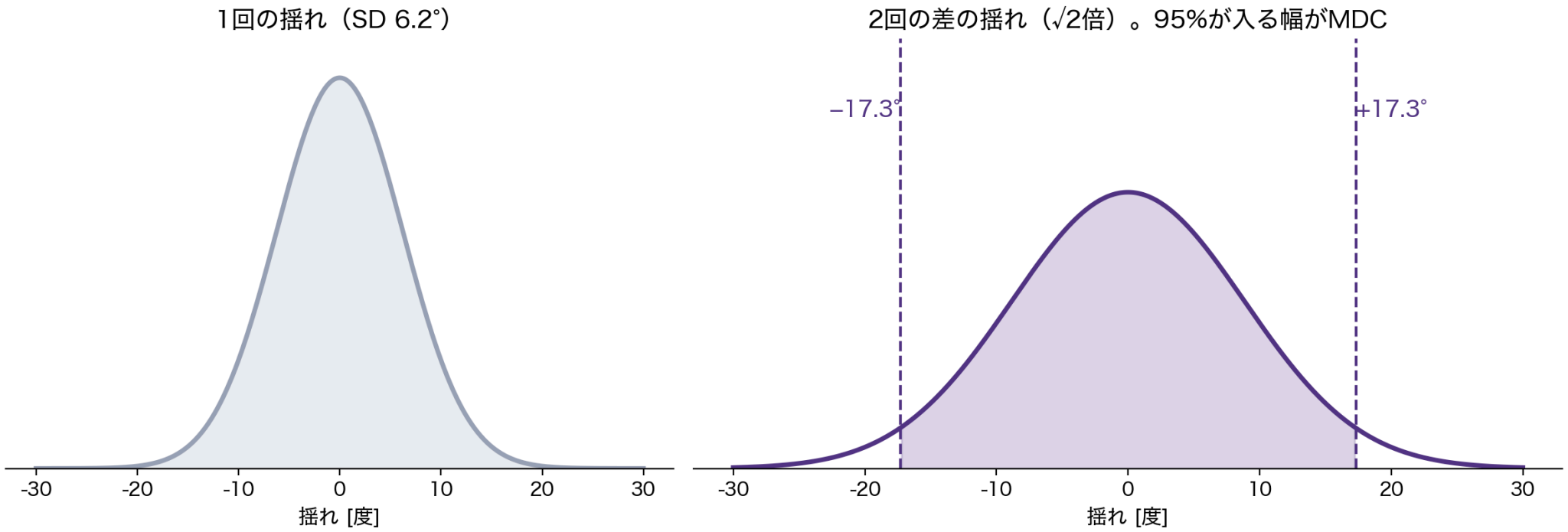
$$\mathrm{MDC}_{95} = 1.96 \times \sqrt{2} \times \mathrm{SEM}$$

- 最小可検変化量（minimal detectable change）。この幅を超えたら、95%の確からしさで「測定の揺れではない」と言える



式を読む

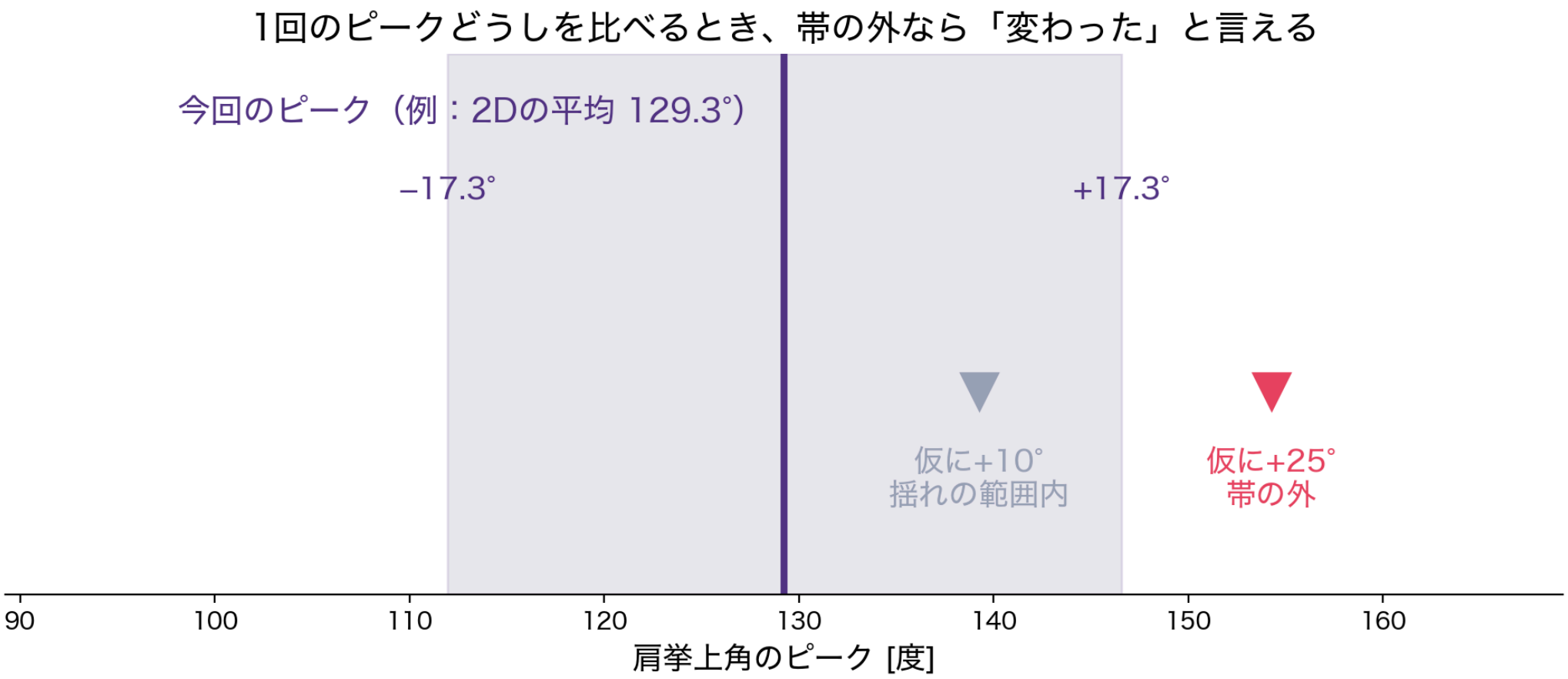
- $\sqrt{2}$：比べるには2回測る。2回とも揺れるので、揺れが2回ぶん入る
- 1.96：揺れの95%が入る幅にするための数



`peak_summary` の `return` の行に、この式がそのまま書いてある

帯の外に出たら変わったと言える

- 1回のピークを次の1回と比べるとき、上下17.3°が揺れの範囲



ゴニオメーターと比べる

測り方	SEM	MDC95
ゴニオメーター（肩屈曲、検者内）	2°	5.5°
今日の動画（2D、5回のピークのSD）	6.2°	17.3°

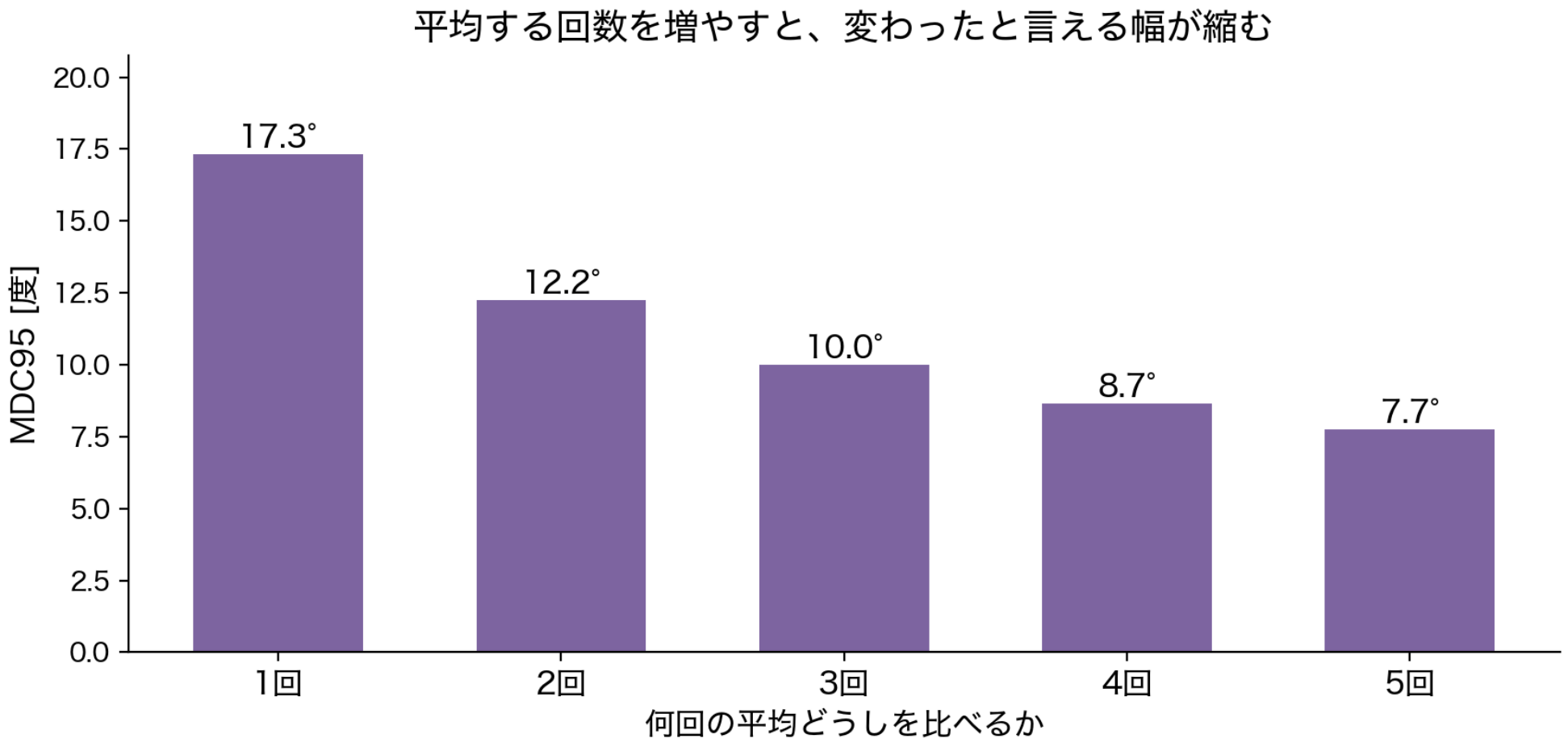
ゴニオメーター：Kolber & Hanney（2012）、健常者30人。MDC95はSEMから上の式で計算

なぜ動画のほうが3倍も大きいのか

ゴニオメーターは「最大まで上げて止めた姿勢」を測る。今日の動画は、最大を狙っていない繰り返しを測っている

揺れは測り方で小さくできる

- 毎回「できるところまで上げて止める」と指示する
- 同じ向き・同じ距離から撮る
- 数回の平均で比べる。5回の平均どうしなら、幅は約7.7°（17.3÷√5）



5. 撮る向きで数字は変わるか

揺れの幅は分かった。では、撮る向きを変えると、数字はどう変わるか

横から撮ったら測れるか？

- 同じ外転を、正面・斜め・横から撮った3本で比べる

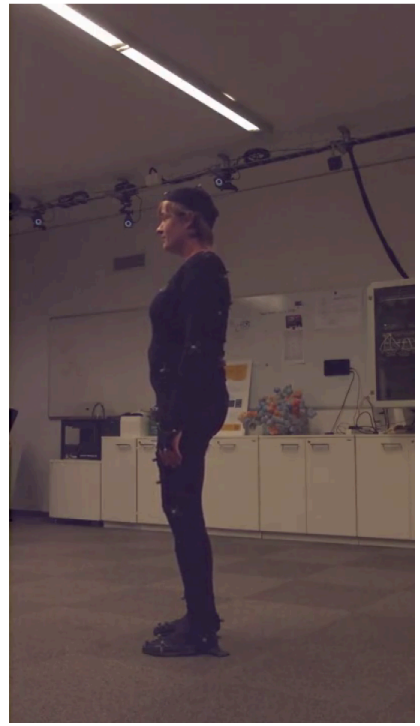
正面



斜め



横



1行で書く繰り返し

- カルテの束を1枚ずつめくり、同じ欄を書き写すのと同じ

```
a2 = np.array([shoulder_elevation(p) for p in p2])
```

- `p2` から1コマずつ `p` として取り出し、肩挙上角にして並べる
- 結果は、実習①で `for` と `.append` を使って作った `angle2d` と同じ

3本で正解と比べる

```
def compare_with_truth(name):
    p2, p3, f = track_pose(name + '.mp4')
    a2 = np.array([shoulder_elevation(p) for p in p2])
    a3 = np.array([shoulder_elevation(p) for p in p3])
    t = np.loadtxt(name + '_truth.csv', delimiter=',', skiprows=1, usecols=2)
    return np.nanmean(np.abs(a2 - t)), np.nanmean(np.abs(a3 - t))

for name in ['front', 'oblique', 'side']:
    d2, d3 = compare_with_truth(name)
    print(name, '2Dの差', round(d2, 1), '3Dの差', round(d3, 1))
```

これまでのセルでしたことを関数にまとめ、3本の動画それぞれに使う

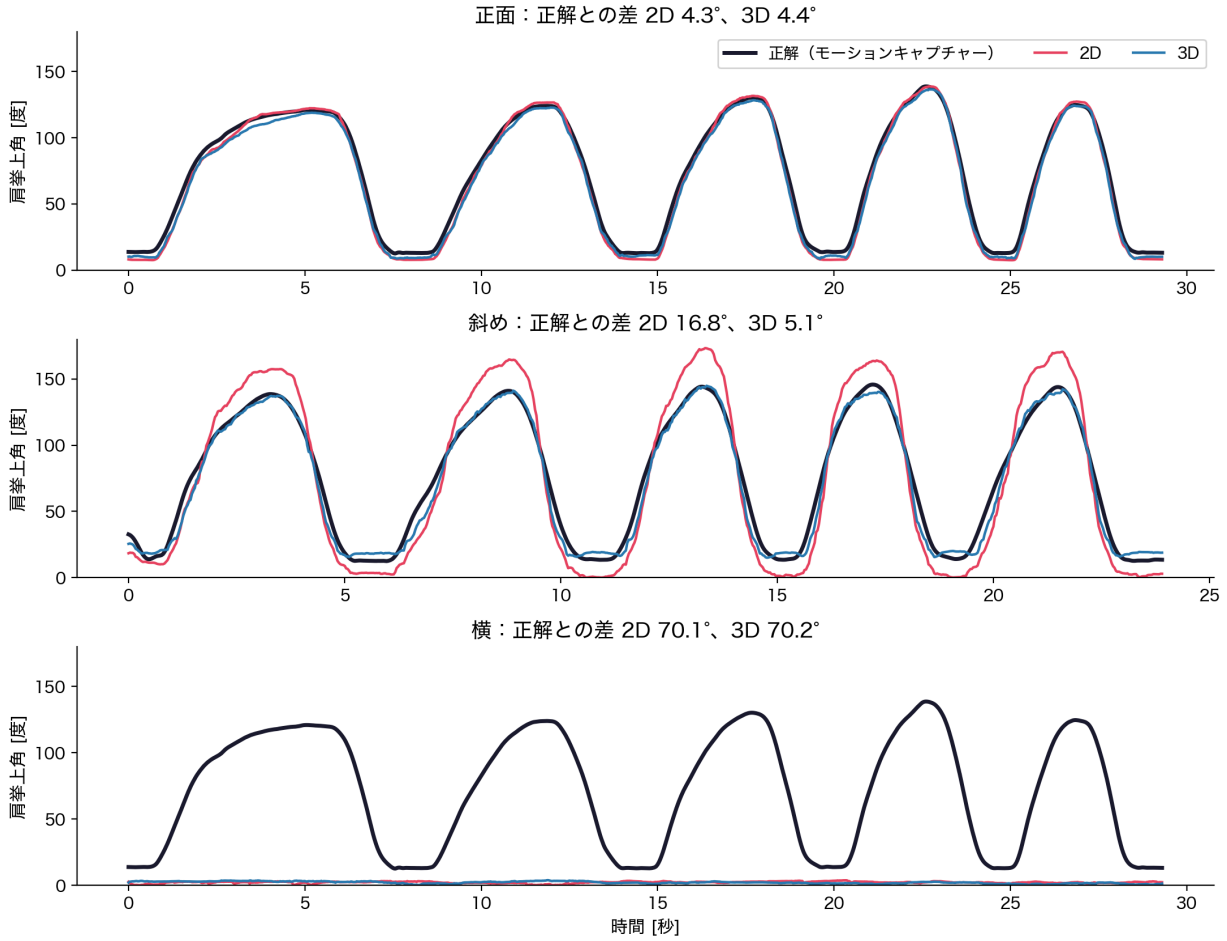
出てくる数

```
front 2Dの差 4.3 3Dの差 4.4
oblique 2Dの差 16.8 3Dの差 5.1
side 2Dの差 70.1 3Dの差 70.2
```

- 正面：2Dも3Dも4°ほど
- 斜め：2Dは17°、3Dは5°
- 横：どちらも70°

向きごとに線を重ねる

- 斜めでは2Dが大きめに出る。横では腕が上がっていないように見える



横からは腕が体の陰に隠れる

- 本人の左側から撮っているので、上げている右腕が体に隠れる

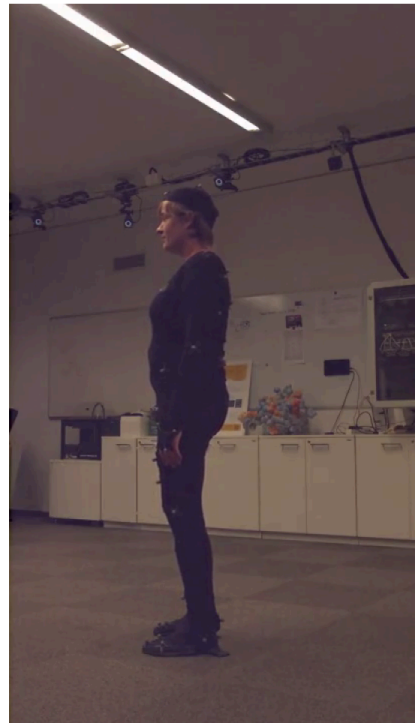
正面



斜め

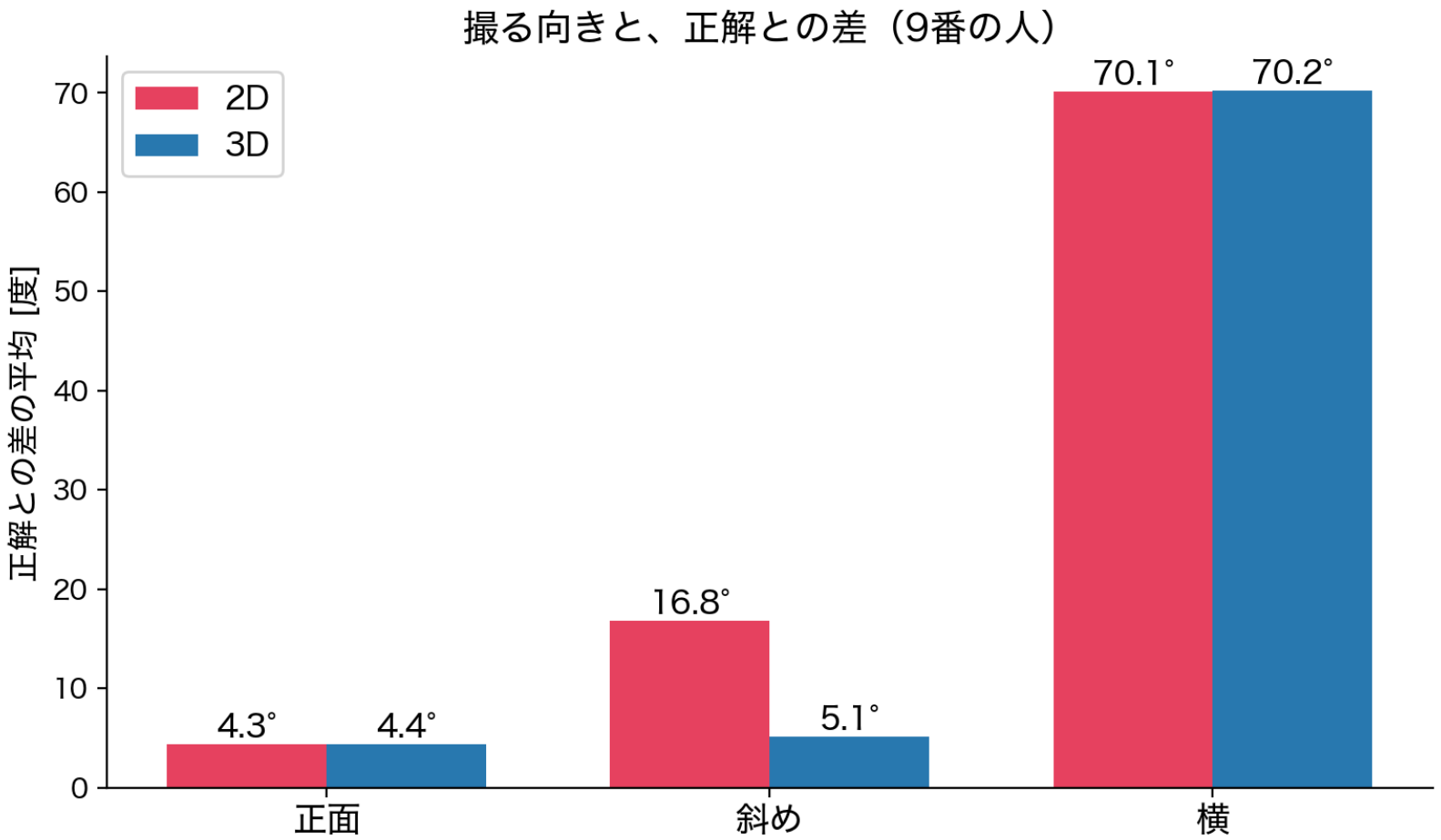


横



斜めでは3Dが助けになる

- 斜めから撮ると動きの面がゆがんで写り、2Dの角度は大きく出る
- 3Dは奥行きを推定しているので、ゆがみを戻せる



正面なら2Dで足りる。斜めなら3D。動きが奥行きの向きになる撮り方では、2Dでも3Dでも正しく測れない

撮る前のチェック

項目	そろえ方
向き	外転は正面から、屈曲は真横から。動きの面とカメラを向かい合わせる
画面	動きの最初から最後まで、全身が入る
服	体の線が分かる服
距離と高さ	毎回同じ場所に、同じ高さで置く

やってみよう：自分の動画

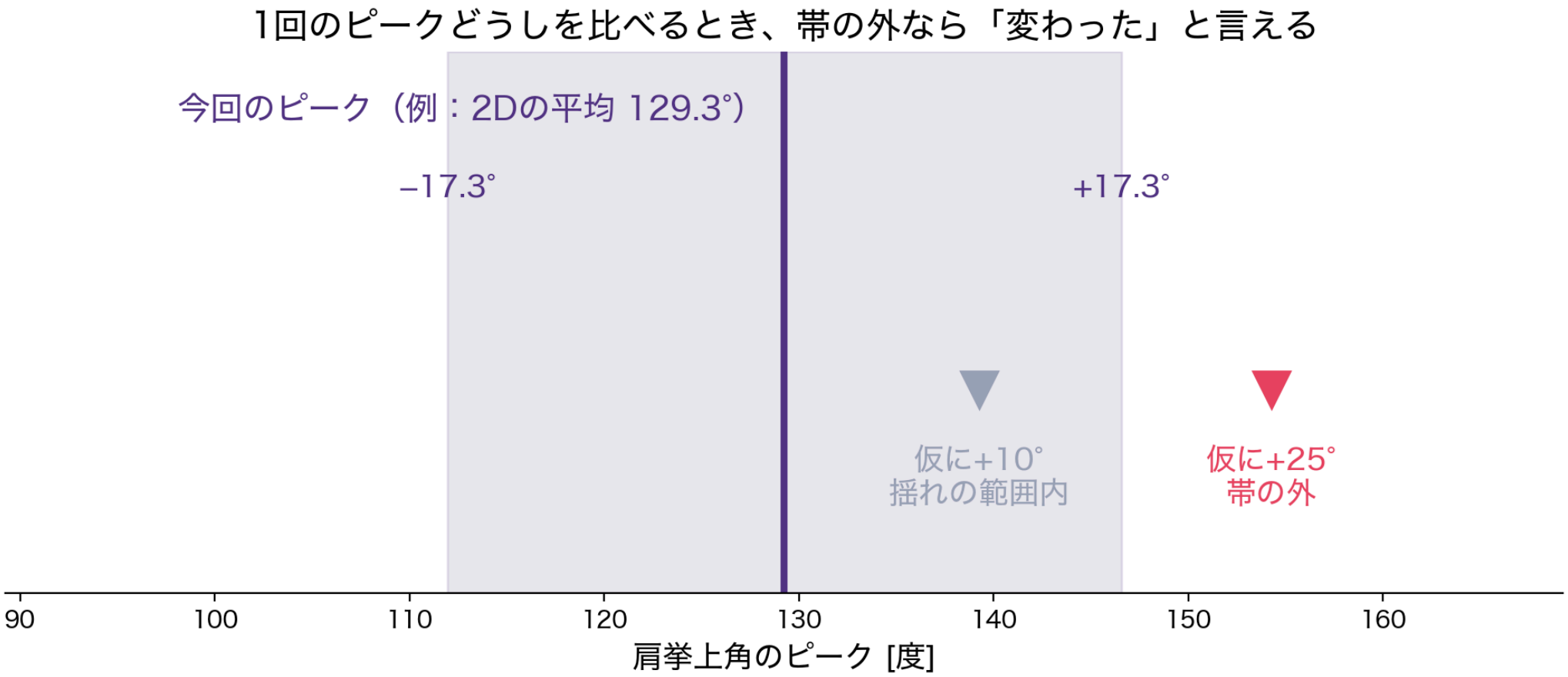
```
from google.colab import files
uploaded = files.upload()
my_file = list(uploaded.keys())[0]
my2d, my3d, my_fps = track_pose(my_file)
my_angle2d = np.array([shoulder_elevation(p) for p in my2d])
my_angle3d = np.array([shoulder_elevation(p) for p in my3d])
my_time = np.arange(len(my2d)) / my_fps
plt.plot(my_time, my_angle2d, label='2D')
plt.plot(my_time, my_angle3d, label='3D')
plt.xlabel('時間 [秒]')
plt.ylabel('肩挙上角 [度]')
plt.legend()
plt.show()
```

撮って、測る

1. 隣の人に、腕を横に5回上げてもらい、正面から全身が入るように撮る
2. ノートの「16. 自分の動画を測る」を押し、動画を選ぶ
3. 線を見て、ノートの「17. 実習①の問い」にメモで答える

良くなったと言うための条件

1. 毎回、同じ撮り方・同じ指示で撮る
2. 1回の値で比べず、数回の平均で比べる
3. 変化が、その撮り方のMDCを超えている

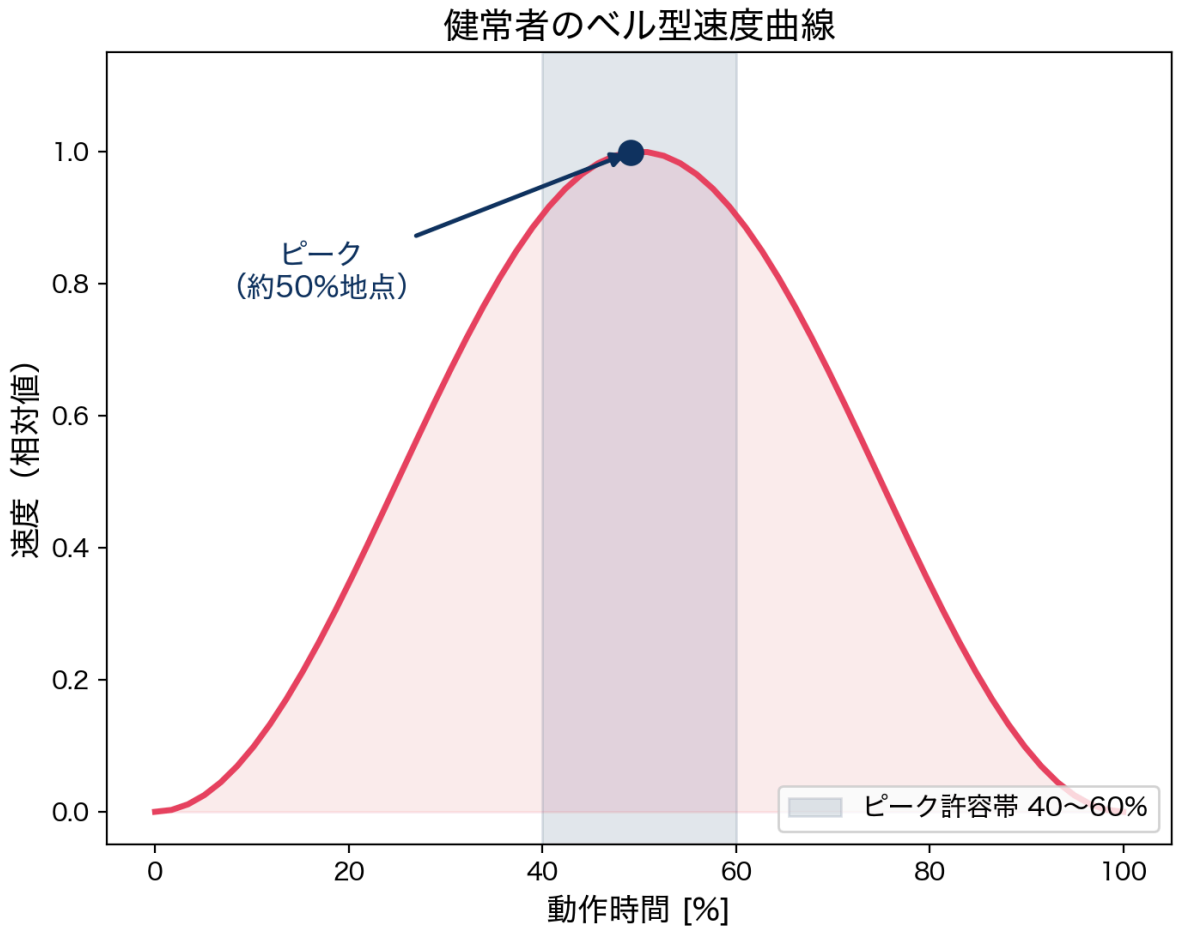


6. どこまで、ではなく、どう動くか

どこまで上がるかは測れた。では、どう動くかは測れるか

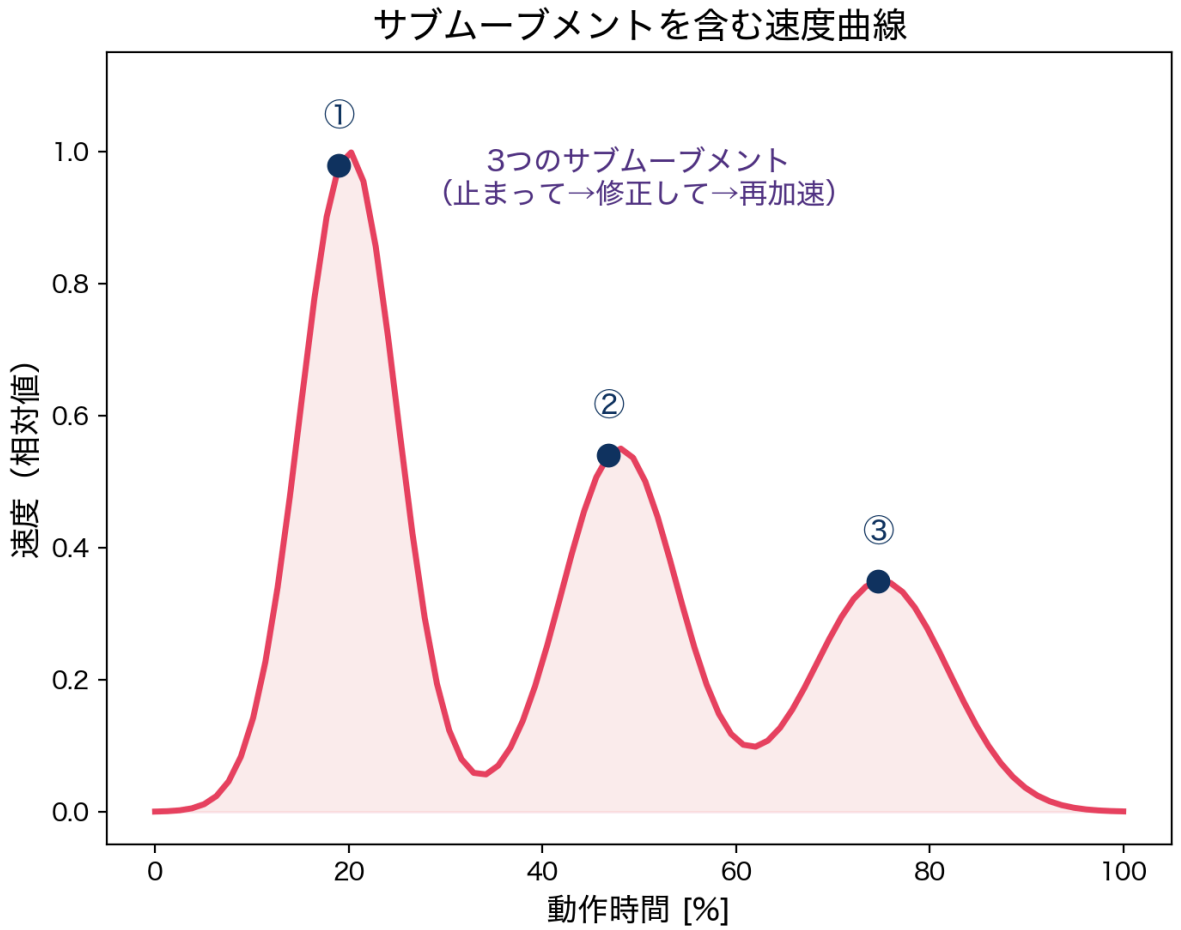
なめらかな動きはベル型

- 手の速さは、すっと上がって、すっと下がる。1つの動きに山は1つ（模式図）



迷うと山が増える

- 止まりかけては動き直すと、山が複数になる。1つ1つの山をサブムーブメントと呼ぶ（模式図）



全部のコマの、1つの点

- 表から縦の1列だけを抜き出すのと同じ。行がコマ、列が点の番号

```
wrist_all = points3d[:, 16]
print('右手首を取り出したコマ数', len(wrist_all))
print('4秒目の右手首', np.round(wrist_all[120], 2))
```

```
右手首を取り出したコマ数 880
4秒目の右手首 [-0.57 -0.68 -0.23]
```

- `:` は「全部」。3つの数は、腰の中央からの横・縦・奥行き（メートル）

速さは距離×コマ数

```
wrist = points3d[:, 16] - (points3d[:, 23] + points3d[:, 24]) / 2
step = np.linalg.norm(np.diff(wrist, axis=0), axis=1)
speed = step * fps
print('速さの最大（m/s）', round(np.nanmax(speed), 2))
```

```
速さの最大（m/s） 1.83
```

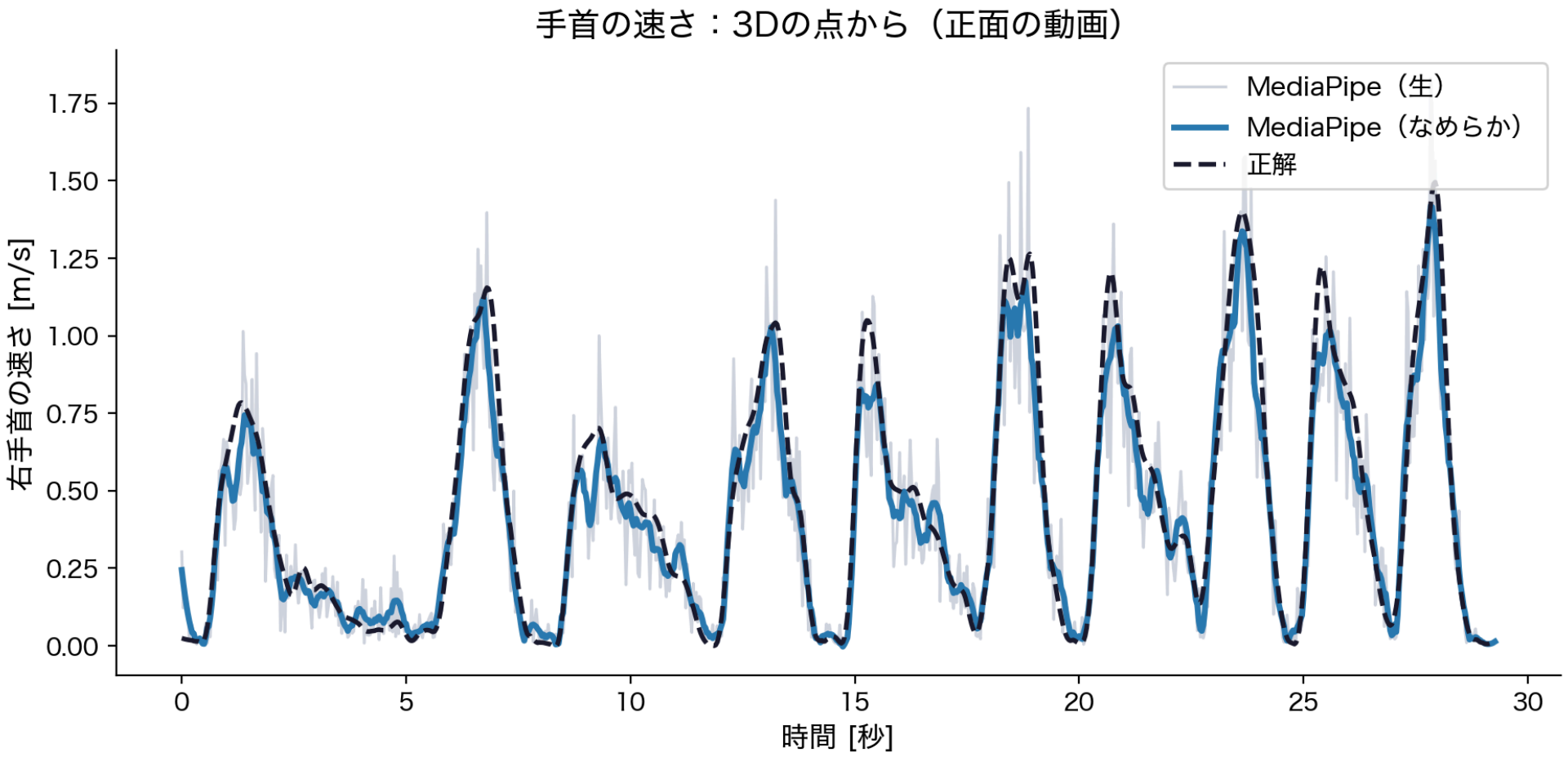
- 1行目：右手首（16番）の、腰の中央からの位置
- 2行目：隣のコマとの差（`diff`）の長さ（`norm`）＝1コマで動いた距離
- 3行目：1コマで動いた距離×1秒あたりのコマ数＝1秒あたりに動く距離

3Dだからm/sで出せる

3Dの点はメートル単位。2Dの点（ピクセル）からは、メートルの速さは出ない

生の速さはぎざぎざ

- 推定した位置がコマごとに少し揺れ、速さにするとその揺れが大きく出る

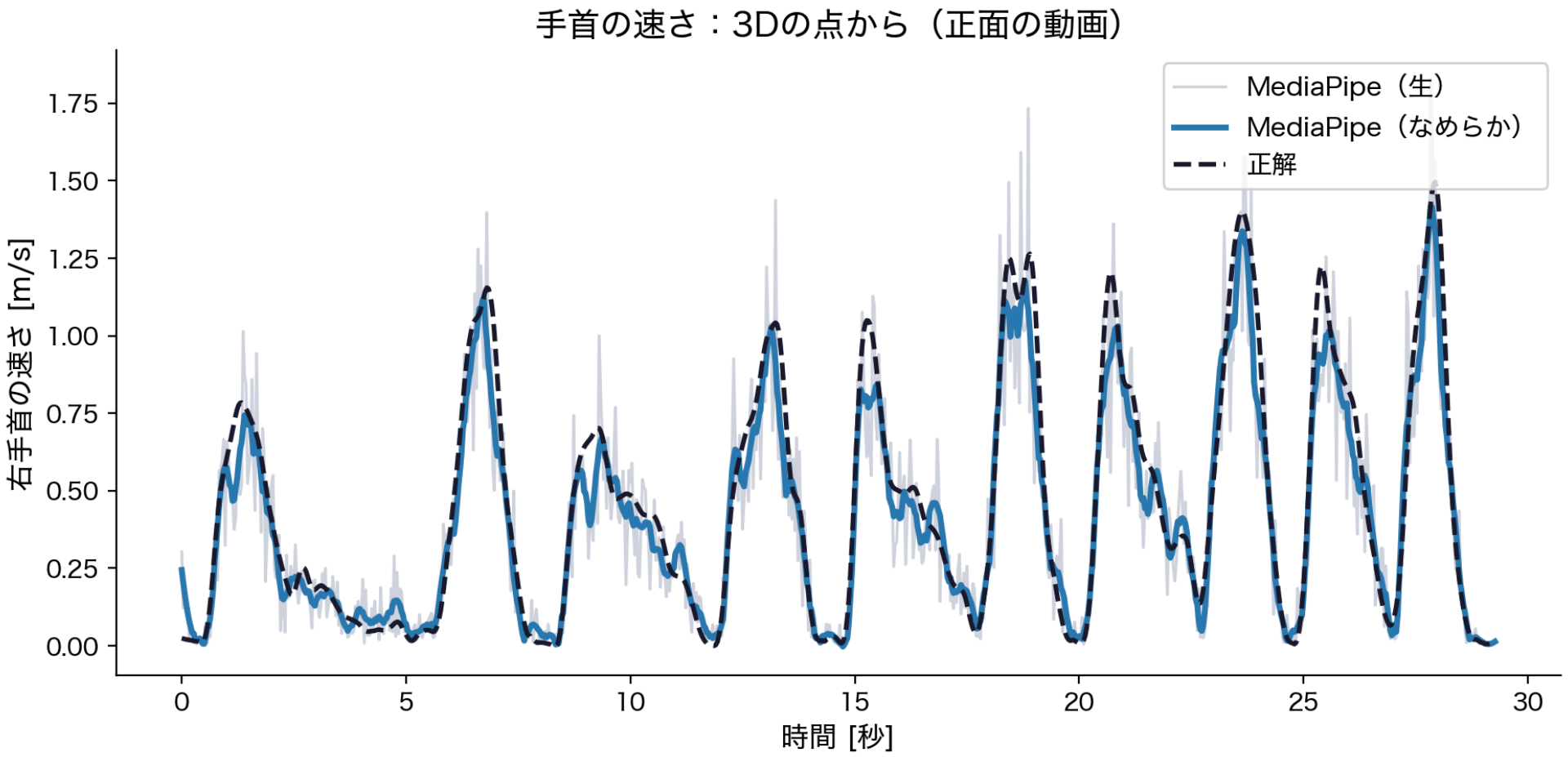


なめらかにして正解と重ねる

```
truth_wrist = np.loadtxt('front_truth_wrist.csv', delimiter=',', skiprows=1,
                         usecols=(2, 3, 4))
truth_speed = np.linalg.norm(np.diff(truth_wrist, axis=0), axis=1) * fps
smooth = savgol_filter(np.nan_to_num(speed), window_length=15, polyorder=2)
truth_smooth = savgol_filter(truth_speed, window_length=15, polyorder=2)
t = np.arange(len(speed)) / fps
plt.plot(t, speed, color='lightgray', label='MediaPipe（生）')
plt.plot(t, smooth, label='MediaPipe（なめらか）')
plt.plot(t, truth_smooth, color='black', label='正解')
plt.xlabel('時間 [秒]')
plt.ylabel('手首の速さ [m/s]')
plt.legend()
plt.show()
```

なめらかにすると正解に重なる

- `savgol_filter` は、前後15コマを使って曲線をなめらかにする関数



山を数える

```
for name, s in [('MediaPipe', smooth), ('正解', truth_smooth)]:
    peaks, _ = find_peaks(s, height=s.max() * 0.1, prominence=s.max() * 0.1)
    print(name, '山の数', len(peaks), 'いちばん速いとき', round(s.max(), 2))
print('MediaPipeと正解の相関', round(np.corrcoef(smooth, truth_smooth)[0, 1], 2))
```

```
MediaPipe 山の数 11 いちばん速いとき 1.42
正解 山の数 10 いちばん速いとき 1.5
MediaPipeと正解の相関 0.98
```

正解の10個は、5回×上げ下げ

腕を1回上げて下ろすと、上げるときと下ろすときで山が2つできる

1つ多い山はどこから？

9秒あたりの1つの山が、MediaPipeでは2つに割れていた。推定の揺れのせいで、山が1つ増えた

速さの形は正解とよく合う。ただし、山の数は推定の揺れに左右されやすい

やってみよう：実習②

1. ノートの「18. 手首の速さを出す」から「21.」までを、上から順に押す
2. 速さの曲線を、正解と重ねて見る
3. ノートの「21.」の問いに、メモで答える

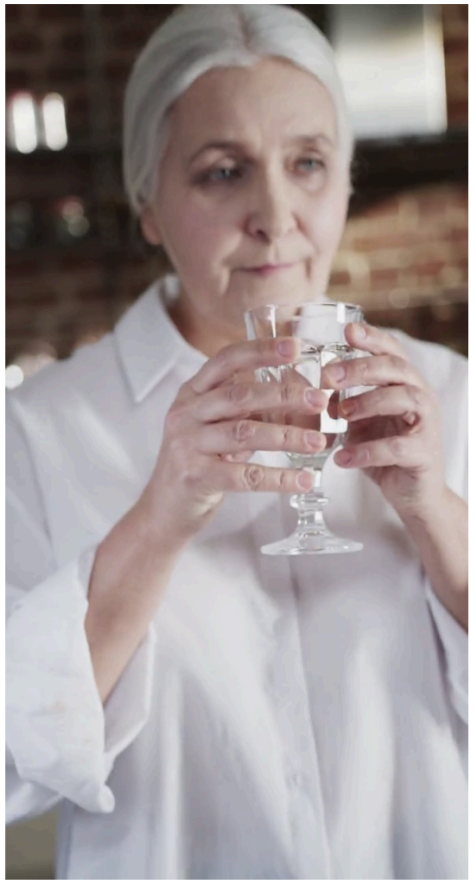
7. 生活の動き：飲水動作を見る

ここまでは、決められた運動（外転）だった。では、生活の中の動きはどう見えるか

グラスを口に運んで飲み、下ろす

- Pexelsで公開されている26秒の動画。モーションキャプチャーの正解はない

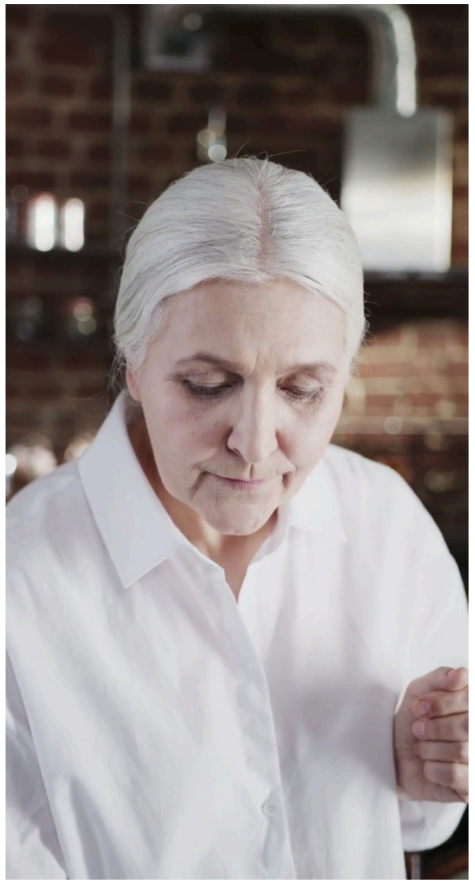
1秒：口元に運んだ



22.5秒：下ろし始め

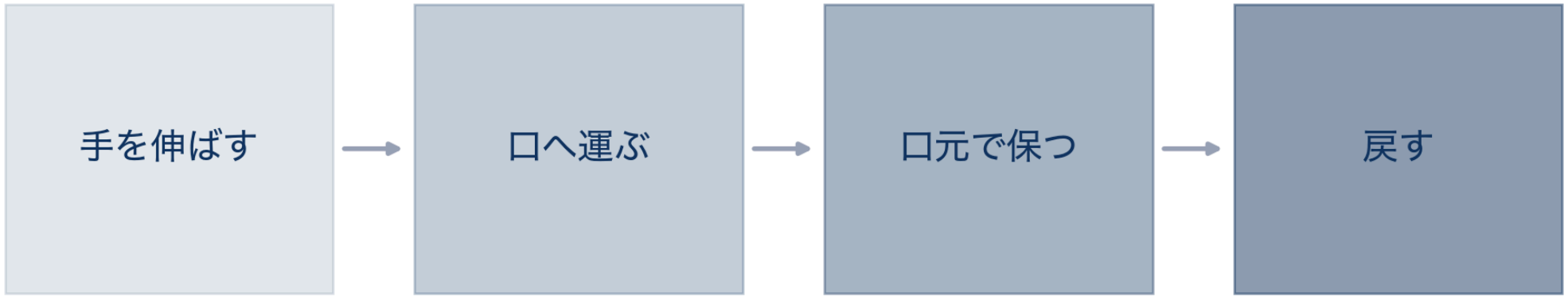


25秒：下ろした



飲水課題は上肢の評価でよく使う

- 1つの課題に、手を伸ばす・運ぶ・口元で保つ・戻すがそろっている
- 脳卒中後の上肢の運動学的評価で、勧められている課題の1つ
- ここでは角度の正しさは問わず、「動作の区切り」を読む



Kwakkelら（2019）、脳卒中後の上肢の動きの質の測り方についての合意（Neurorehabil Neural Repair 33:951–958）

手の高さと肘の角度を測る

```
d2, d3, dfps = track_pose('drink.mp4')
mid_shoulder = (d3[:, 11] + d3[:, 12]) / 2
hand = d3[:, 16] - mid_shoulder
height = -hand[:, 1]
elbow = np.array([180 - angle_between_vectors(p[12] - p[14], p[16] - p[14])
                  for p in d3])
print('コマ数', len(d3))
print('手の高さ 最小', round(np.nanmin(height), 2), '最大', round(np.nanmax(height), 2))
print('肘の屈曲角 最小', round(np.nanmin(elbow)), '最大', round(np.nanmax(elbow)))
```

```
コマ数 650
手の高さ 最小 -0.41 最大 -0.03
肘の屈曲角 最小 25 最大 98
```

3行ずつ読む

- 腰が写っていないので、右手首（16番）を両肩の中点からの位置にする
- MediaPipeの3Dの点は下向きが正。`-`を付けて上向きを正にする
- 肘の屈曲角は、肘を軸に肘→肩と肘→手首のなす角を180°から引く

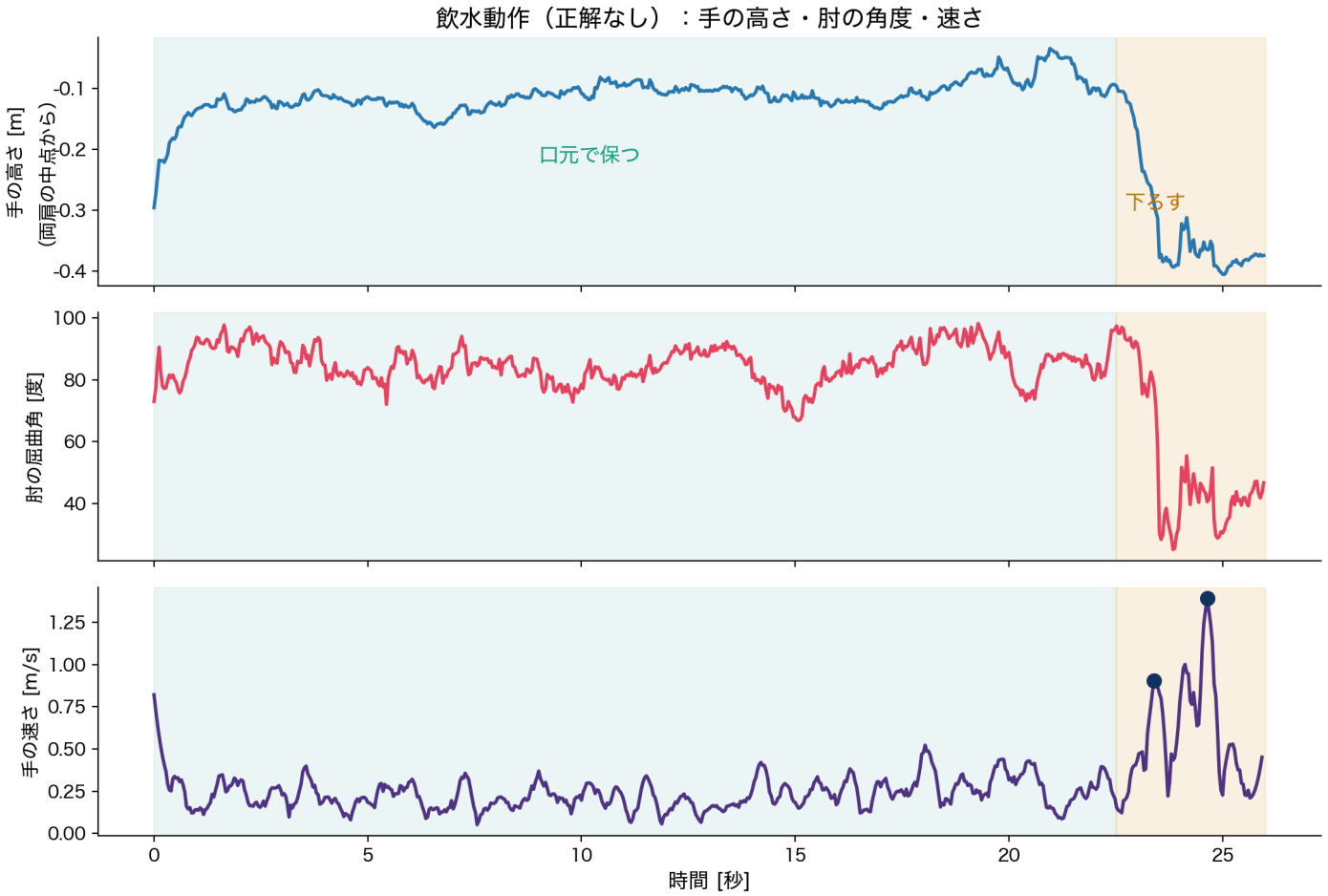
グラフで区切りを読む

```
t = np.arange(len(d3)) / dfps
fig, axes = plt.subplots(2, 1, sharex=True)
axes[0].plot(t, height)
axes[0].set_ylabel('手の高さ [m]')
axes[1].plot(t, elbow)
axes[1].set_ylabel('肘の屈曲角 [度]')
axes[1].set_xlabel('時間 [秒]')
plt.show()
```

`plt.subplots(2, 1)` は、グラフを縦に2つ並べる関数

口元で保ち、22.5秒から下ろす

- 手の高さは約20秒間ほぼ一定、肘は80〜95°。下ろすと肘は30°前後に伸びる



速さの山を数える

```
dspeed = np.linalg.norm(np.diff(hand, axis=0), axis=1) * dfps
dsmooth = savgol_filter(np.nan_to_num(dspeed), window_length=15, polyorder=2)
peaks, _ = find_peaks(dsmooth, height=dsmooth.max() * 0.4,
                      prominence=dsmooth.max() * 0.4)
print('山の数', len(peaks), '山の時刻（秒）', np.round(peaks / dfps, 1))
```

```
山の数 2 山の時刻（秒） [23.4 24.6]
```

持ち上げる山はどこに？

山は下ろすとき（23〜25秒）の2つだけ。動画が持ち上げる途中から始まるので、持ち上げの山は取れない。どこから撮り始めるかで、測れるものが決まる

やってみよう：自分のリーチ（任意）

```
from google.colab import files
uploaded = files.upload()
my_reach = list(uploaded.keys())[0]
r2, r3, rfps = track_pose(my_reach)
r_hand = r3[:, 16] - (r3[:, 11] + r3[:, 12]) / 2
r_speed = np.linalg.norm(np.diff(r_hand, axis=0), axis=1) * rfps
r_smooth = savgol_filter(np.nan_to_num(r_speed), window_length=15, polyorder=2)
plt.plot(np.arange(len(r_smooth)) / rfps, r_smooth)
plt.xlabel('時間 [秒]')
plt.ylabel('右手首の速さ [m/s]')
plt.show()
```

時間があれば、撮って測る

1. 隣の人に、机のコップに手を伸ばして口に運び、戻す動きを3回してもらう
2. 動き出す前から撮り始め、上半身が入るように撮る
3. ノートの「26. 自分のリーチを撮って測る（任意）」を押し、1往復の山の数を見る

8. 手指を見る

肩・肘・手首の次は、手。体のモデルとは別の、手のモデルを使う

手の21点

- 0が手首、5〜8が人差し指（5が付け根、6がPIP、7がDIP、8が指先）

手の21点（できた回、手首がいちばん高いコマ）



手のモデルを呼ぶ関数

関数	役割	入力	出力
`track_hand`	全コマで手の21点を出す	動画の名前	手の2Dの点、1秒のコマ数

ノートの「28.」。中身は`track_pose`と同じ形で、読み飛ばしてよい

指の曲がりと手首の高さ

```
def hand_measures(name):
    points, fps = track_hand(name + '.mp4')
    pip = np.array([180 - angle_between_vectors(p[5] - p[6], p[7] - p[6]) for p in points])
    size = np.nanmedian(np.linalg.norm(points[:, 9] - points[:, 0], axis=1))
    rise = -(points[:, 0, 1] - np.nanmedian(points[:15, 0, 1])) / size
    print(name, 'コマ数', len(points), '手が見つかったコマの割合',
          round(np.mean(~np.isnan(points[:, 0, 0])), 2))
    print('  人差し指のPIPの屈曲（中央値）', round(np.nanmedian(pip), 1),
          ' 手首の上がり方の最大（手の大きさ）', round(np.nanmax(rise), 2))
    return pip, rise, fps

ok_pip, ok_rise, hand_fps = hand_measures('lift_complete')
ng_pip, ng_rise, hand_fps = hand_measures('lift_incomplete')
```

中身は読み飛ばしてよい。入力は動画の名前、出力は指の曲がり・手首の上がり方・1秒のコマ数

2つのかたまりで読む

1. 人差し指のPIPの屈曲角：PIPを軸に、PIP→付け根とPIP→DIPのなす角を180°から引く
2. 手首の高さ：最初の15コマからの上がり方を、手の大きさ（手首→中指の付け根）を1として表す

手の21点（できた回、手首がいちばん高いコマ）



手の大きさで割るので、カメラからの距離が違っても比べられる

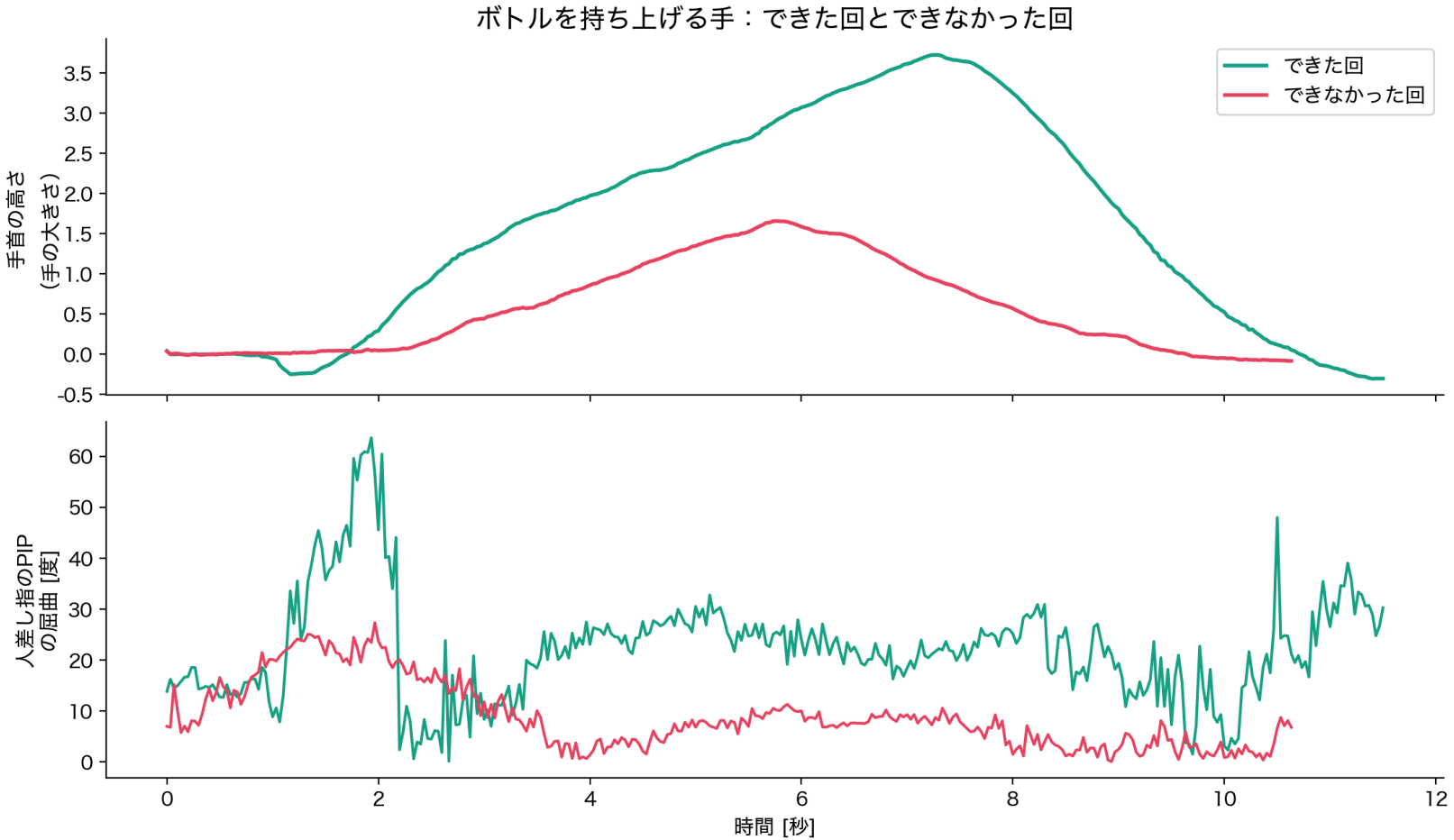
出てくる数

```
lift_complete コマ数 346 手が見つかったコマの割合 1.0
  人差し指のPIPの屈曲（中央値） 21.8  手首の上がり方の最大（手の大きさ） 3.73
lift_incomplete コマ数 320 手が見つかったコマの割合 1.0
  人差し指のPIPの屈曲（中央値） 7.6  手首の上がり方の最大（手の大きさ） 1.66
```

Mendeley Dataの上肢運動の動画。健常な人が「できた回」と「できなかった回」を演じたもの（CC BY 4.0）

できなかった回は持ち上がらず、握りも浅い

- 手首は半分ほどしか上がらず、指の曲がりも小さい



この動画には正解がない。差は見えても、角度が正しいかは確かめられていない

やってみよう：手指

1. ノートの「28.」から「30.」まで、上から順に押す
2. 手首の高さと指の曲がりのグラフを見比べる
3. 「31. 実習④の問い」にメモで答える

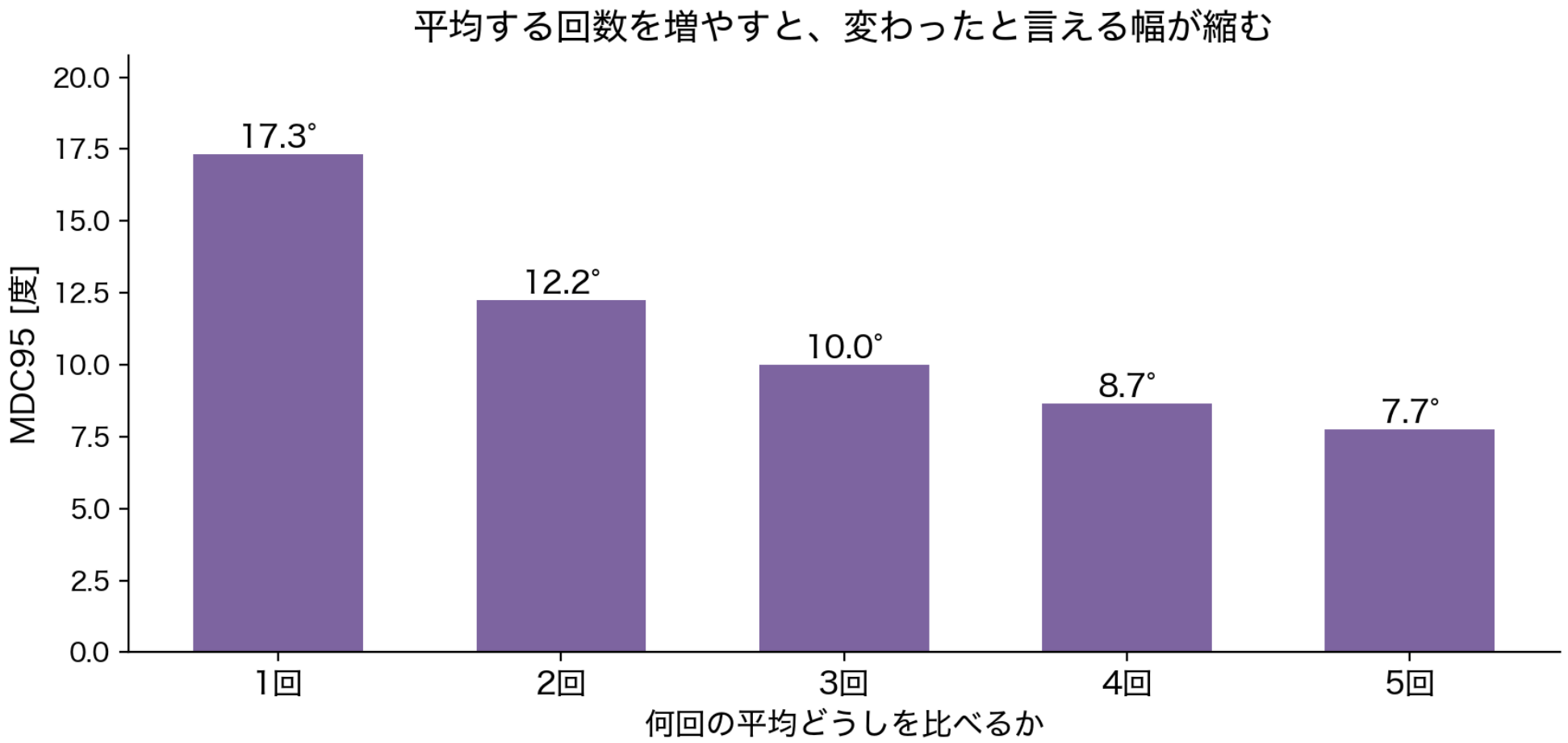
まとめ

今日の問い

スマホ動画1本から、この上肢は良くなったと言えるか

言えること

- 正面から撮れば、MediaPipeの2Dでも正解との差は4°ほど
- 今日の撮り方なら、1回のピークが17°を超えて変われば「変わった」と言える
- 撮る向きと指示をそろえ、数回の平均で比べれば、この幅は小さくできる



まだ言えないこと

- 確かめたのは健常な人の外転。屈曲や患者さんの動きは確かめていない
- 日をまたいだ揺れは測っていない。幅はもっと広いかもしれない
- 飲水動作と手指は正解がなく、角度の正しさは確かめていない

先の研究：点数が頭打ちの後も

	中身
対象	神経疾患の患者20人、47回の評価、ARATの1174課題
方法	臨床のARATの評価を、そのままマーカーレスで撮った
結果	ARATの点が満点近くで止まった後も、動きの指標は改善をとらえた

Ungerら（2026）、arXiv:2607.23608。査読前の論文

持ち帰る1文

同じ撮り方で何度か測り、揺れの幅を超えた変化だけを「良くなった」と言う

配布物

もの	使い方
本編のスライド（PDF）	今日のスライド
発展編のスライド（PDF）	挙上面と挙上角・代償・SAM 3D Bodyなどの最新モデル
実習ノート（本編）	Colabで開き、上から全部押す
発展のノート	発展編A・Bの実習（任意）
発展のノートC	SAM 3D BodyをColabのT4 GPUで動かす（任意、Hugging Faceの申請が要る）
配布データ	ノートが自動で読み込む（REHAB24-6、Pexels、Mendeley Data）